

Databricks Certified Data Engineer Associate

প্র্যাকটিস প্রশ্ন ও বাংলা ব্যাখ্যা

মোট প্রশ্ন: 45

প্রতিটি প্রশ্নে: মূল প্রশ্ন (ইংরেজি) → সঠিক উত্তর → বিস্তারিত ব্যাখ্যা → মনে রাখার উপায়

1. Databricks Repos ও CI/CD
2. Delta Lake কী
3. CREATE OR REPLACE TABLE (খালি Delta table)
4. INSERT INTO দিয়ে append
5. Z-Ordering
6. CREATE DATABASE IF NOT EXISTS ... LOCATION
7. Managed table
8. View (data কপি ছাড়া)
9. Parquet cache সমস্যা → Delta
10. CTAS-এ schema
11. Overwrite বনাম Delete + Recreate
12. SELECT DISTINCT
13. INNER JOIN
14. explode()
15. Struct field: dot notation
16. PySpark দিয়ে dynamic table নাম
17. createOrReplaceTempView()
18. Python f-string
19. readStream (Auto Loader)
20. trigger(once=True)
21. Auto Loader: শুধু নতুন file
22. format("cloudFiles")
23. Bronze table source
24. Silver table source
25. Bronze → Silver hop
26. DLT: table dependency
27. DLT pipeline: Jobs page
28. CREATE LIVE TABLE
29. DLT Expectations (warn)
30. DLT: Triggered + Development mode
31. Task-level retry policy
32. Task dependency (linear)
33. Job config: JSON
34. SQL endpoint: cluster size
35. SQL query schedule
36. Databricks SQL Alert
37. Dashboard refresh দেরি
38. GRANT SELECT
39. GRANT ALL PRIVILEGES
40. Lakehouse বনাম Data Warehouse
41. Control plane বনাম Data plane
42. Unstructured data + ACID
43. Job cluster বনাম All-purpose cluster
44. Data Explorer দিয়ে permission
45. Real-time coauthoring

Question 1

QUESTION

Which of the following describes how **Databricks Git Folders** can help facilitate CI/CD workflows on the **Databricks Data Intelligence Platform**?

- A. **Databricks Git Folders** can facilitate the pull request, review, and approval process before merging branches
- B. **Databricks Git Folders** can merge changes from a secondary Git branch into a main Git branch
- C. **Databricks Git Folders** can be used to design, develop, and trigger Git automation pipelines
- D. **Databricks Git Folders** can store the single-source-of-truth Git repository
- E. **Databricks Git Folders** can commit or push code changes to trigger a CI/CD process

✓ সঠিক উত্তর: E

ব্যাখ্যা

Databricks Repos হলো Databricks workspace-এর ভেতরে একটা **Git client**। এটা বাইরের Git provider-এর (GitHub, GitLab, Azure DevOps, Bitbucket) সাথে যুক্ত থাকে। Repos দিয়ে এই কাজগুলো করা যায়:

- Repository **clone** করা
- Branch তৈরি করা ও এক branch থেকে আরেকটায় যাওয়া (**checkout**)
- Code-এর পরিবর্তন **commit** ও **push** করা
- Remote থেকে **pull** করা

CI/CD workflow-তে Repos-এর মূল কাজ হলো developer-এর লেখা code Git provider পর্যন্ত পৌঁছে দেওয়া। Developer Databricks-এ notebook বা code লিখে Repos থেকে commit/push করেন। সেই push-এর ফলে Git provider-এ সেট করা CI/CD pipeline (যেমন GitHub Actions, Azure Pipelines, Jenkins) **নিজে থেকেই trigger হয়**। তাই **E** সঠিক।

✗ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Pull request, code review আর approval হয় Git provider-এ (যেমন GitHub-এ)। Databricks Repos-এ এই সুবিধা নেই।
B	পরীক্ষার প্রেক্ষিতে, feature branch থেকে main branch-এ merge সাধারণত হয় Git provider-এ, PR approve হওয়ার পর। CI/CD workflow-এ এটা Repos-এর কাজ ধরা হয় না।
C	Automation pipeline (GitHub Actions, Azure DevOps Pipelines, Jenkins ইত্যাদি) CI/CD tool-এ design করা ও চালানো হয়। Repos দিয়ে pipeline design বা develop করা যায় না।
D	Single source of truth হলো remote Git repository , যেটা Git provider-এ থাকে। Databricks Repos তার শুধু একটা local working copy (clone)।

মনে রাখার উপায়

Databricks Repos = Databricks-এর ভেতরের Git client (clone, branch, commit, push, pull) **Git Provider**
= PR, review, merge, source of truth CI/CD Tool = Pipeline design ও execution

সংক্ষেপে: Databricks Repos থেকে **commit/push করলে CI/CD process trigger হয়**। তাই উত্তর **E**।

Question 2

QUESTION

Which of the following statements describes Delta Lake?

- A. Delta Lake is an open source analytics engine used for big data workloads.
- B. Delta Lake is an open format storage layer that delivers reliability, security, and performance.
- C. Delta Lake is an open source platform to help manage the complete machine learning lifecycle.
- D. Delta Lake is an open source data storage format for distributed data.
- E. Delta Lake is an open format storage layer that processes data

✓ সঠিক উত্তর: B

ব্যাখ্যা

Delta Lake হলো একটি **open format storage layer**। এটা data lake-এর (যেমন S3, ADLS, GCS) উপরে বসে এবং **Databricks Lakehouse Platform-এর ভিত্তি** হিসেবে কাজ করে। Databricks-এর official সংজ্ঞাও প্রায় এটাই: Delta Lake এমন একটি open format storage layer যা data lake-এ **reliability, security এবং performance** নিয়ে আসে।

এই তিনটা গুণ Delta Lake কীভাবে দেয়:

- **Reliability (নির্ভরযোগ্যতা)**
 - **ACID transaction:** একসাথে অনেকে read/write করলেও data নষ্ট হয় না
 - **Schema enforcement ও evolution:** ভুল format-এর data ঢুকতে বাধা দেয়
 - **Time travel:** data-র আগের version দেখা বা সেখানে ফিরে যাওয়া যায়
 - **Transaction log (`_delta_log`):** প্রতিটা পরিবর্তনের রেকর্ড থাকে
- **Security (নিরাপত্তা)**
 - Unity Catalog-এর সাথে মিলে fine-grained access control ও auditing
- **Performance (দক্ষতা)**
 - Data skipping, Z-Ordering, file compaction (`OPTIMIZE`), caching ইত্যাদি

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	এটা Apache Spark -এর বর্ণনা। Spark হলো analytics/processing engine, Delta Lake engine নয়।
C	এটা MLflow -এর বর্ণনা। MLflow দিয়ে ML lifecycle (experiment tracking, model registry, deployment) সামলানো হয়।
D	Delta Lake শুধু একটা storage format নয়। এর ভেতরে data Parquet file হিসেবে থাকে, আর তার উপরে transaction log দিয়ে ACID, time travel-এর মতো সুবিধা দেওয়া হয়। তাই একে " storage layer " বলা হয়। এই সংজ্ঞাটা অসম্পূর্ণ, এবং এটা অনেকটা Parquet-এর বর্ণনার মতো শোনায়।
E	Delta Lake নিজে data process করে না । Processing করে Spark-এর মতো compute engine। Delta Lake শুধু data সংরক্ষণ ও সামলানোর layer।

🔑 মনে রাখার উপায়

Delta Lake = Storage layer → **Reliability + Security + Performance** **Apache Spark** = Processing/analytics engine **MLflow** = ML lifecycle management **Parquet** = শুধু file format (Delta Lake-এর ভেতরে এটাই ব্যবহার হয়)

সংক্ষেপে: Delta Lake হলো **open format storage layer যা reliability, security 3 performance দেয়**। তাই উত্তর **B**।

Question 3

QUESTION

A data architect has determined that a table of the following format is necessary:

id	birthDate	avgRating
a1	1990-01-06	5.5
a2	1974-11-21	7.1
...	...	...

Which of the following code blocks uses SQL DDL commands to create an empty Delta table in the above format regardless of whether a table already exists with this name?

A.

```
CREATE OR REPLACE TABLE table_name AS
SELECT
  id STRING,
  birthDate DATE,
  avgRating FLOAT
USING DELTA
```

B.

```
CREATE OR REPLACE TABLE table_name (
  id STRING,
  birthDate DATE,
  avgRating FLOAT
)
```

C.

```
CREATE TABLE IF NOT EXISTS table_name (
  id STRING,
  birthDate DATE,
  avgRating FLOAT
)
```

D.

```
CREATE TABLE table_name AS
SELECT
  id STRING,
  birthDate DATE,
  avgRating FLOAT
```

E.

```
CREATE OR REPLACE TABLE table_name WITH COLUMNS (  
  id STRING,  
  birthDate DATE,  
  avgRating FLOAT  
) USING DELTA
```

✓ সঠিক উত্তর: B

ব্যাখ্যা

প্রশ্নে তিনটি শর্ত আছে। Option B তিনটিই পূরণ করে:

1. **খালি table (empty table):** CREATE TABLE table_name (column definitions) syntax-এ শুধু schema ঠিক হয়, কোনো data ঢাকে না। তাই table খালি থাকে।
2. **Table আগে থেকে থাকুক বা না থাকুক:** CREATE OR REPLACE দুই অবস্থাই সামলায়।
 - Table না থাকলে → নতুন table তৈরি করে
 - Table থাকলে → সেটাকে নতুন schema দিয়ে replace করে
3. **Delta table:** Databricks-এ Delta Lake হলো default table format। তাই USING DELTA না লিখলেও table টা Delta table হিসেবেই তৈরি হয়।

✗ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	CREATE TABLE ... AS SELECT (CTAS) একটা query-র ফলাফল থেকে table বানায়, তাই এটা খালি table-এর জন্য নয়। তাছাড়া SELECT -এর ভেতরে id STRING -এর মতো data type লেখার নিয়ম নেই, আর USING DELTA AS SELECT -এর আগে বসে, পরে নয়। তাই syntax ভুল।
C	CREATE TABLE IF NOT EXISTS table না থাকলে বানায়। কিন্তু table আগে থেকে থাকলে কিছুই করে না, পুরনো table যেমন ছিল তেমনই থাকে। ফলে "আগে থেকে থাকুক বা না থাকুক" খালি table পাওয়ার শর্ত পূরণ হয় না।
D	এটাও CTAS, আর SELECT -এর ভেতরে data type দেওয়া ভুল syntax। তাছাড়া OR REPLACE না থাকায় table আগে থেকে থাকলে error দেবে।
E	SQL-এ WITH COLUMNS বলে কোনো keyword নেই। তাই এটা ভুল syntax।

🔑 মনে রাখার উপায়

Syntax	কী করে
<code>CREATE TABLE t (...)</code>	Table বানায়; আগে থেকে থাকলে error
<code>CREATE TABLE IF NOT EXISTS t (...)</code>	না থাকলে বানায়; থাকলে কিছু করে না
<code>CREATE OR REPLACE TABLE t (...)</code>	না থাকলে বানায়; থাকলে replace করে ✅
<code>CREATE TABLE t AS SELECT ... (CTAS)</code>	Query-র data সহ table বানায় (খালি নয়)

💡 Databricks-এ **default format = Delta**, তাই `USING DELTA` লেখা ঐচ্ছিক।

সংক্ষেপে: খালি Delta table বানাতে এবং থাকলে replace করতে লাগে **`CREATE OR REPLACE TABLE` + column definition**। তাই উত্তর **B**।

Question 4

QUESTION

Which of the following SQL keywords can be used to append new rows to an existing Delta table?

- A. UPDATE
- B. COPY
- C. INSERT INTO
- D. DELETE
- E. UNION

✓ সঠিক উত্তর: C. INSERT INTO

ব্যাখ্যা

`INSERT INTO` কোনো table-এ **নতুন row যোগ করে**। Table-এ আগে থেকে যে data আছে তাতে হাত দেয় না, নতুন data শুধু শেষে জুড়ে দেয়। একেই **append** বলে।

```
-- সরাসরি value দিয়ে row যোগ করা
INSERT INTO students VALUES
  ('a3', '1995-03-15', 8.2),
  ('a4', '1988-07-30', 6.9);

-- অন্য table বা query-র ফলাফল যোগ করা
INSERT INTO students
SELECT * FROM new_students;
```

Delta table-এ প্রতিটা `INSERT INTO` একটা **ACID transaction** হিসেবে চলে, আর এর রেকর্ড transaction log-এ একটা নতুন version হিসেবে থাকে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. UPDATE	আগে থেকে থাকা row-এর value পরিবর্তন করে , নতুন row যোগ করে না। যেমন: <code>UPDATE students SET avgRating = 9.0 WHERE id = 'a1'</code>
B. COPY	শুধু COPY বলে কোনো append keyword নেই। Databricks-এ আছে COPY INTO , যা file (cloud storage) থেকে data load করে। কিন্তু প্রশ্নে শুধু COPY বলা হয়েছে, আর table-এ সাধারণভাবে row append করার standard keyword হলো INSERT INTO ।
D. DELETE	Table থেকে row মুছে ফেলে , যোগ করে না।
E. UNION	দুটো query-র ফলাফল একসাথে মিলিয়ে দেখায় । কিন্তু table-এ কোনো data লেখে না, শুধু একটা result set ফেরত দেয়।

🔑 মনে রাখার উপায়

Keyword	কাজ
INSERT INTO	নতুন row যোগ (append) ✅
INSERT OVERWRITE	পুরনো data মুছে নতুন data দিয়ে প্রতিস্থাপন
UPDATE	আগে থেকে থাকা row পরিবর্তন
DELETE	row মুছে ফেলা
MERGE INTO	শর্ত অনুযায়ী insert / update / delete (upsert)
COPY INTO	File থেকে data load (idempotent: একই file দ্বিতীয়বার load করে না)
UNION	Query-র ফলাফল মেলানো (table-এ লেখে না)

সংক্ষেপে: Delta table-এ নতুন row append করার keyword হলো **INSERT INTO**। তাই উত্তর **C**।

Question 5

QUESTION

A data engineering team needs to query a Delta table to extract rows that all meet the same condition. However, the team has noticed that the query is running slowly. The team has already tuned the size of the data files. Upon investigating, the team has concluded that the rows meeting the condition are sparsely located throughout each of the data files.

Based on the scenario, which of the following optimization techniques could speed up the query?

- A. Data skipping
- B. Z-Ordering
- C. Bin-packing
- D. Write as a Parquet file
- E. Tuning the file size

✓ সঠিক উত্তর: B. Z-Ordering

ব্যাখ্যা

সমস্যাটা কী? দরকারি row গুলো সব file-এ অল্প অল্প করে ছড়ানো। তাই query-কে প্রায় প্রতিটা file পড়তে হচ্ছে, কারণ সব file-এই কিছু না কিছু মিলে যাওয়া row আছে। কোনো file বাদ দেওয়া যাচ্ছে না, আর এ কারণেই query ধীর।

Z-Ordering কীভাবে সমাধান করে? Z-Ordering নির্দিষ্ট column-এর মান অনুযায়ী data সাজিয়ে কাছাকাছি মানের row গুলোকে একই file-এ রাখে (co-locate করে)। ফলে:

- শর্ত পূরণ করা row গুলো অল্প কয়েকটা file-এ জমা হয়
- প্রতিটা file-এর min/max statistics আরও নির্দিষ্ট হয়ে যায়
- Query চলার সময় Delta Lake অপ্রয়োজনীয় file গুলো বাদ দিতে পারে (data skipping কার্যকর হয়)
- কম file পড়তে হয়, তাই query দ্রুত হয়

```
OPTIMIZE my_table  
ZORDER BY (condition_column);
```

উদাহরণ:

অবস্থা	File 1	File 2	File 3	File 4	কয়টা file পড়তে হয়
Z-Order এর আগে	✓ কিছু row	✓ কিছু row	✓ কিছু row	✓ কিছু row	৪টা
Z-Order এর পরে	✓ সব row	✗	✗	✗	মাত্র ১টা

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. Data skipping	Delta Lake-এ data skipping নিজে থেকেই চালু থাকে । কিন্তু row যখন সব file-এ ছড়ানো, তখন প্রতিটা file-এর min/max range-এর মধ্যেই শর্তের মান পড়ে, তাই কোনো file বাদ দেওয়া যায় না। Data skipping কাজে লাগাতে হলে আগে Z-Ordering দিয়ে data সাজাতে হয়।
C. Bin-packing	<code>OPTIMIZE</code> (ZORDER ছাড়া) অনেক ছোট file মিলিয়ে বড় file বানায় (compaction)। এটা "small file problem" মেটায়, কিন্তু row-গুলোকে column-এর মান অনুযায়ী সাজায় না। তাছাড়া file size আগেই tune করা হয়েছে।
D. Parquet হিসেবে লেখা	Delta table-এর ভেতরে data আগে থেকেই Parquet file হিসেবে থাকে। সাধারণ Parquet-এ লিখলে উল্টো Delta-র সুবিধাগুলো (transaction log, statistics) হারাতে হবে।
E. File size tune করা	প্রশ্নেই বলা আছে team আগেই file size tune করেছে । সমস্যাটা file-এর size নিয়ে নয়, file-এর ভেতরে data কীভাবে সাজানো তা নিয়ে।

🔑 মনে রাখার উপায়

সমস্যা	সমাধান
অনেক ছোট file (small file problem)	Bin-packing / <code>OPTIMIZE</code>
দরকারি row সব file-এ ছড়ানো (sparse)	Z-Ordering / <code>OPTIMIZE ... ZORDER BY</code> ✅
Query-তে অপ্রয়োজনীয় file এড়ানো	Data skipping (নিজে থেকে চলে, Z-Order থাকলে আরও ভালো কাজ করে)

💡 **Z-Ordering** = একই রকম data এক জায়গায় আনা, যাতে data skipping ভালোভাবে কাজ করতে পারে।

সংক্ষেপে: row যখন সব file-এ ছড়িয়ে থাকে, তখন **Z-Ordering** সেগুলোকে অল্প কয়েকটা file-এ এনে query দ্রুত করে। তাই উত্তর **B**।

Question 6

QUESTION

A data engineer needs to create a database called customer360 at the location /customer/customer360. The data engineer is unsure if one of their colleagues has already created the database.

Which of the following commands should the data engineer run to complete this task?

- A. `CREATE DATABASE customer360 LOCATION '/customer/customer360';`
- B. `CREATE DATABASE IF NOT EXISTS customer360;`
- C. `CREATE DATABASE IF NOT EXISTS customer360 LOCATION '/customer/customer360';`
- D. `CREATE DATABASE IF NOT EXISTS customer360 DELTA LOCATION '/customer/customer360';`
- E. `CREATE DATABASE customer360 DELTA LOCATION '/customer/customer360';`

✓ সঠিক উত্তর: C

```
CREATE DATABASE IF NOT EXISTS customer360
LOCATION '/customer/customer360';
```

ব্যাখ্যা

প্রশ্নে দুটো শর্ত আছে, আর Option C দুটোই পূরণ করে:

1. **Database আগে থেকে থাকতে পারে** → `IF NOT EXISTS`
 - Database না থাকলে → নতুন database তৈরি করবে
 - Database থাকলে → কোনো error দেবে না, কিছু না করে চুপচাপ এগিয়ে যাবে
2. **নির্দিষ্ট location-এ বানাতে হবে** → `LOCATION '/customer/customer360'`
 - এতে database-এর data default location-এ না রেখে নির্দিষ্ট path-এ রাখা হয়

💡 Databricks-এ `DATABASE` আর `SCHEMA` একই জিনিস। তাই `CREATE SCHEMA IF NOT EXISTS ...` লিখলেও একই কাজ হবে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	IF NOT EXISTS নেই। সহকর্মী database টা আগে বানিয়ে থাকলে command টা error দেবে (database already exists)।
B	IF NOT EXISTS আছে, কিন্তু LOCATION নেই । Database টা default location-এ তৈরি হবে, /customer/customer360 -এ নয়।
D	DELTA LOCATION বলে কোনো syntax নেই। তাই এটা ভুল syntax । Delta হলো table format, database-এর জন্য format বলে দেওয়ার দরকার নেই।
E	IF NOT EXISTS নেই, আর DELTA LOCATION ভুল syntax । দুই দিক থেকেই ভুল।

🔑 মনে রাখার উপায়

প্রয়োজন	ব্যবহার করতে হবে
আগে থেকে থাকলে error এড়ানো	IF NOT EXISTS
নির্দিষ্ট path-এ database রাখা	LOCATION '<path>'
Database-এর জন্য DELTA keyword	❌ লাগে না (Delta হলো table format, আর default)

সংক্ষেপে: **IF NOT EXISTS** দিয়ে error এড়ানো আর **LOCATION** দিয়ে path ঠিক করা, এই দুটো একসাথে আছে শুধু Option **C**-তে। তাই উত্তর **C**।

Question 7

QUESTION

A junior data engineer needs to create a Spark SQL table `my_table` for which Spark manages both the data and the metadata. The metadata and data should also be stored in the Databricks Filesystem (DBFS).

Which of the following commands should a senior data engineer share with the junior data engineer to complete this task?

- A. `CREATE TABLE my_table (id STRING, value STRING) USING org.apache.spark.sql.parquet OPTIONS (PATH "storage-path");`
- B. `CREATE MANAGED TABLE my_table (id STRING, value STRING) USING org.apache.spark.sql.parquet OPTIONS (PATH "storage-path");`
- C. `CREATE MANAGED TABLE my_table (id STRING, value STRING);`
- D. `CREATE TABLE my_table (id STRING, value STRING) USING DBFS;`
- E. `CREATE TABLE my_table (id STRING, value STRING);`

✓ সঠিক উত্তর: E

```
CREATE TABLE my_table (id STRING, value STRING);
```

ব্যাখ্যা

Spark-এ table দুই রকমের হয়:

ধরন	Data কে manage করে	Metadata কে manage করে	Table DROP করলে
Managed table	Spark	Spark	Data + metadata দুটোই মুছে যায়
External (unmanaged) table	User (নিজের দেওয়া path)	Spark	শুধু metadata মুছে যায়, data থেকে যায়

প্রশ্নে চাওয়া হয়েছে **Spark যেন data আর metadata দুটোই manage করে**, অর্থাৎ একটা **managed table**।

Managed table কীভাবে বানায়? `CREATE TABLE` -এর সাথে `LOCATION` বা `OPTIONS (PATH ...)` না দিলে Spark নিজে থেকেই managed table বানায়। Data রাখা হয় default warehouse location-এ (যেমন `dbfs:/user/hive/warehouse/my_table`), যা **DBFS-এর ভেতরেই**। তাই Option E দুটো শর্তই পূরণ করে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	OPTIONS (PATH "storage-path") দিয়ে নিজের path দেওয়া হয়েছে, তাই এটা external (unmanaged) table হবে। Data Spark manage করবে না।
B	SQL-এ MANAGED বলে কোনো keyword নেই, তাই syntax ভুল। তার উপর PATH দেওয়ায় এটা external table-ই হতো।
C	এখানেও CREATE MANAGED TABLE syntax ব্যবহার হয়েছে, যার অস্তিত্ব নেই। Table managed হয় LOCATION / PATH না দিলেই, আলাদা কোনো keyword লাগে না।
D	USING -এর পরে data format বসে (DELTA, PARQUET, CSV ইত্যাদি)। DBFS কোনো format নয় , এটা একটা file system। তাই syntax ভুল।

🔑 মনে রাখার উপায়

```
-- ✅ Managed table (Spark data + metadata দুটোই manage করে)
CREATE TABLE my_table (id STRING, value STRING);

-- External table (data user-এর দেওয়া path-এ থাকে)
CREATE TABLE my_table (id STRING, value STRING)
LOCATION '/mnt/some/path';
```

💡 Path/LOCATION নেই → Managed table 💡 Path/LOCATION আছে → External table 💡 Managed table বানানোর জন্য Databricks SQL-এ **CREATE MANAGED TABLE** বলে কোনো keyword নেই

সংক্ষেপে: কোনো path ছাড়া সাধারণ **CREATE TABLE** লিখলেই managed table তৈরি হয় এবং data DBFS-এর default location-এ থাকে। তাই উত্তর **E**।

Question 8

QUESTION

A data engineer wants to create a relational object by pulling data from two tables. The relational object must be used by other data engineers in other sessions. In order to save on storage costs, the data engineer wants to avoid copying and storing physical data.

Which of the following relational objects should the data engineer create?

- A. View
- B. Temporary view
- C. Delta Table
- D. Database
- E. Spark SQL Table

✓ সঠিক উত্তর: A. View

ব্যাখ্যা

প্রশ্নে তিনটা শর্ত আছে, আর **View** তিনটাই পূরণ করে:

1. **দুটো table থেকে data নেওয়া** → View একটা query-র উপর তৈরি হয়, তাই JOIN দিয়ে একাধিক table মেলানো যায়।
2. **অন্য session থেকেও ব্যবহার করা যাবে** → সাধারণ View **metastore-এ save থাকে** (persistent)। তাই অন্য user বা অন্য session থেকেও এটা পাওয়া যায়।
3. **Physical data কপি হবে না** → View শুধু একটা **saved query**। এটা কোনো data জমা রাখে না। যখনই View-তে query চালানো হয়, তখন মূল table থেকে data পড়া হয়। তাই বাড়তি storage খরচ হয় না।

```
CREATE VIEW customer_orders AS
SELECT c.customer_id, c.name, o.order_id, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B. Temporary view	Data কপি করে না ঠিকই, কিন্তু এটা শুধু যে session-এ বানানো হয়েছে সেই session-এই থাকে। Session শেষ হলে মুছে যায়, আর অন্য session থেকে দেখা যায় না।
C. Delta Table	Table-এ physical data জমা থাকে । দুটো table থেকে data নিয়ে নতুন table বানালে (যেমন CTAS দিয়ে) data কপি হবে, ফলে storage খরচ বাড়বে।
D. Database	Database হলো table/view রাখার একটা container (namespace) । এটা দুটো table থেকে data টেনে আনার মতো object নয়।
E. Spark SQL Table	এটাও physical data জমা রাখে (managed বা external, দুই ক্ষেত্রেই data file থাকে)। তাই storage বাঁচে না।

🔑 মনে রাখার উপায়

Object	Physical data জমা রাখে?	অন্য session থেকে পাওয়া যায়?
View	❌ না	✅ হ্যাঁ ✅
Temporary view	❌ না	❌ না (শুধু নিজের session)
Global temporary view	❌ না	⚠️ একই cluster-এর অন্য session থেকে (global_temp দিয়ে), cluster restart হলে মুছে যায়
Table (Delta / Spark SQL)	✅ হ্যাঁ	✅ হ্যাঁ

💡 **View = saved query** (data নেই, শুধু query-র সংজ্ঞা) 💡 **Table = আসল data**

সংক্ষেপে: data কপি না করে, অন্য session-এও ব্যবহার করা যায় এমন object হলো **View**। তাই উত্তর **A**।

Question 9

QUESTION

A data engineering team has created a series of tables using Parquet data stored in an external system. The team is noticing that after appending new rows to the data in the external system, their queries within Databricks are not returning the new rows. They identify the caching of the previous data as the cause of this issue.

Which of the following approaches will ensure that the data returned by queries is always up-to-date?

- A. The tables should be converted to the Delta format
- B. The tables should be stored in a cloud-based external system
- C. The tables should be refreshed in the writing cluster before the next query is run
- D. The tables should be altered to include metadata to not cache
- E. The tables should be updated before the next query is run

✓ সঠিক উত্তর: A. Delta format-এ convert করা

ব্যাখ্যা

সমস্যাটা কেন হচ্ছে? সাধারণ Parquet table-এ কোন কোন file আছে তার তালিকা Spark একবার পড়ে **cache করে রাখে**। পরে বাইরে থেকে নতুন file যোগ হলে Spark সেটা নিজে থেকে জানতে পারে না। তাই পুরনো cache থেকে পুরনো data-ই দেখায়।

Delta Lake কীভাবে সমাধান করে? Delta table-এ থাকে একটি **transaction log** (`_delta_log`)। প্রতিটা পরিবর্তন (insert, update, delete) এই log-এ একটি নতুন version হিসেবে লেখা হয়।

- প্রতিটা query চালানোর সময় Delta Lake **transaction log দেখে সর্বশেষ version খুঁজে নেয়**
- ফলে কোন file গুলো এখন বৈধ, সেটা সবসময় ঠিকঠাক জানা থাকে
- তাই **query সবসময় up-to-date data ফেরত দেয়**, cache-এর কারণে পুরনো data দেখায় না

```
-- Parquet table-কে Delta-তে convert করা  
CONVERT TO DELTA parquet.`/path/to/parquet/data`;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	Data কোথায় রাখা আছে (cloud বা অন্য কোথাও) তাতে কিছু বদলায় না। সমস্যাটা storage-এর জায়গা নিয়ে নয়, cache-এর কারণে Parquet-এর পুরনো file তালিকা নিয়ে।
C	REFRESH TABLE চালালে কিছুক্ষণের জন্য সমস্যা মেটে, কিন্তু প্রতিবার নিজে হাতে চালাতে হবে । আর refresh করতে হয় query চালানো cluster-এ , writing cluster-এ নয়। এতে "সবসময় up-to-date" নিশ্চিত হয় না।
D	Table-এ "cache না করার" metadata যোগ করার মতো কোনো standard সুবিধা নেই। এটা কোনো বাস্তব সমাধান নয় ।
E	Table "update" করলে (data বদলালে) cache-এর সমস্যা মেটে না। এটা ভুল ধারণা।

🔑 মনে রাখার উপায়

Format	নতুন data কীভাবে জানে	সবসময় up-to-date?
Parquet	File তালিকা cache করে রাখে, নিজে হাতে REFRESH TABLE চালাতে হয়	❌ না
Delta	প্রতিটা query-তে transaction log দেখে সর্বশেষ version পড়ে	✅ হ্যাঁ

💡 **Transaction log = Delta-র "সত্যের উৎস"**। এটা সবসময় বলে দেয় table-এর সর্বশেষ অবস্থা কী।

সংক্ষেপে: Delta Lake-এর **transaction log** নিশ্চিত করে যে প্রতিটা query সর্বশেষ data দেখে। তাই উত্তর **A**।

Question 10

QUESTION

A table `customerLocations` exists with the following schema:

```
id STRING,  
date STRING,  
city STRING,  
country STRING
```

A senior data engineer wants to create a new table from this table using the following command:

```
CREATE TABLE customersPerCountry AS  
SELECT country,  
       COUNT(*) AS customers  
FROM customerLocations  
GROUP BY country;
```

A junior data engineer asks why the schema is not being declared for the new table.

Which of the following responses explains why declaring the schema is not necessary?

- A. CREATE TABLE AS SELECT statements adopt schema details from the source table and query.
- B. CREATE TABLE AS SELECT statements infer the schema by scanning the data.
- C. CREATE TABLE AS SELECT statements result in tables where schemas are optional.
- D. CREATE TABLE AS SELECT statements assign all columns the type STRING.
- E. CREATE TABLE AS SELECT statements result in tables that do not support schemas.

✓ সঠিক উত্তর: A

ব্যাখ্যা

CTAS (CREATE TABLE AS SELECT) নতুন table-এর schema নিজে থেকেই ঠিক করে নেয়। Schema আসে দুই জায়গা থেকে:

1. **Source table থেকে** → `country` column-এর type source table-এ `STRING`, তাই নতুন table-এও `STRING`
2. **Query থেকে** → `COUNT(*)` সবসময় একটা সংখ্যা ফেরত দেয়, তাই `customers` column-এর type হবে `BIGINT`

ফলে নতুন table-এর schema দাঁড়ায়:

```
country    STRING  
customers  BIGINT
```

Spark query টা বিশ্লেষণ করেই (query plan থেকে) প্রতিটা column-এর নাম আর type জেনে যায়। তাই আলাদা করে schema লেখার দরকার হয় না।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	CTAS data scan করে schema অনুমান করে না । Schema আসে source table-এর metadata আর query-র গঠন থেকে। Data scan করে schema infer করা হয় CSV/JSON-এর মতো file পড়ার সময় (inferSchema)।
C	Table-এ schema ঐচ্ছিক নয় । প্রতিটা table-এর একটা নির্দিষ্ট schema থাকে। CTAS-এ শুধু schema নিজে থেকে ঠিক হয়ে যায়, হাতে লিখতে হয় না।
D	সব column STRING হয় না। উপরের উদাহরণেই customers হবে BIGINT । Type আসে source আর query থেকে।
E	CTAS দিয়ে বানানো table অবশ্যই schema support করে । Delta table-এ schema enforcement-ও চালু থাকে।

🔑 মনে রাখার উপায়

পদ্ধতি	Schema কীভাবে আসে
<code>CREATE TABLE t (col TYPE, ...)</code>	হাতে লিখে দিতে হয়
<code>CREATE TABLE t AS SELECT ... (CTAS)</code>	Source table + query থেকে নিজে থেকে নেয় ✅
CSV/JSON file পড়া (inferSchema)	Data scan করে অনুমান করে

💡 CTAS-এ column-এর type বদলাতে চাইলে query-র ভেতরেই **CAST** ব্যবহার করতে হয়। যেমন: `CAST(date AS DATE) AS date`

সংক্ষেপে: CTAS **source table আর query থেকে schema নিয়ে নেয়**, তাই আলাদা করে লেখার দরকার নেই। উত্তর **A**।

Question 11

QUESTION

A data engineer is overwriting data in a table by deleting the table and recreating the table. Another data engineer suggests that this is inefficient and the table should simply be overwritten instead.

Which of the following reasons to overwrite the table instead of deleting and recreating the table is incorrect?

- A. Overwriting a table is efficient because no files need to be deleted.
- B. Overwriting a table results in a clean table history for logging and audit purposes.
- C. Overwriting a table maintains the old version of the table for Time Travel.
- D. Overwriting a table is an atomic operation and will not leave the table in an unfinished state.
- E. Overwriting a table allows for concurrent queries to be completed while in progress.

✓ সঠিক উত্তর (ভুল যুক্তি): B

⚠ খেয়াল করুন: প্রশ্নে **ভুল (incorrect)** যুক্তিটা খুঁজতে বলা হয়েছে।

ব্যাখ্যা

B কেন ভুল? Delta table overwrite করলে **history মুছে যায় না, "পরিষ্কার" হয় না**। বরং overwrite-টা transaction log-এ **আরেকটা নতুন version হিসেবে যোগ হয়**। আগের সব version-এর রেকর্ড history-তে থেকে যায়।

```
DESCRIBE HISTORY my_table;
```

version	operation
0	CREATE TABLE
1	WRITE (append)
2	CREATE OR REPLACE / WRITE (overwrite) ← নতুন version যোগ হলো

তাই "পরিষ্কার history" পাওয়ার কথাটা ভুল। আসলে পুরো history রাখা থাকে, আর **audit-এর জন্য এটাই সুবিধা**। উল্টো দিকে, table **delete করে আবার বানালে** পুরনো history হারিয়ে যায়।

✓ বাকি যুক্তিগুলো কেন সঠিক

Option	কেন সঠিক
A	Overwrite করার সময় Delta পুরনো file গুলো সাথে সাথে delete করে না । শুধু transaction log-এ সেগুলোকে "আর বৈধ নয়" বলে চিহ্নিত করে, আর নতুন file লেখে। এতে সময় বাঁচে। (পুরনো file পরে VACUUM দিয়ে মুছে ফেলা যায়।)
C	পুরনো file থেকে যায় বলে Time Travel দিয়ে আগের version দেখা যায়: <code>SELECT * FROM my_table VERSION AS OF 1</code>
D	Delta-র প্রতিটা write একটা ACID transaction । Overwrite হয় পুরোটা সফল হবে, নয়তো কিছুই বদলাবে না। মাঝপথে fail করলেও table আধাআধি অবস্থায় থাকে না।
E	Overwrite চলার সময় অন্য query গুলো table-এর আগের version পড়তে থাকে (snapshot isolation)। তাই তারা বাধা ছাড়াই শেষ হতে পারে। কিন্তু table delete করলে চলমান query fail করবে।

🔑 মনে রাখার উপায়

বিষয়	Delete + Recreate	Overwrite ✓
History	✗ হারিয়ে যায়	✓ থাকে (নতুন version যোগ হয়)
Time Travel	✗ সম্ভব নয়	✓ সম্ভব
Atomic	✗ না (মাঝখানে table থাকে না)	✓ হ্যাঁ
Concurrent query	✗ fail করতে পারে	✓ চলতে থাকে
File delete	সাথে সাথে	✗ লাগে না (পরে VACUUM)

```
-- Overwrite করার উপায়
CREATE OR REPLACE TABLE my_table AS SELECT ...;
-- অথবা
INSERT OVERWRITE my_table SELECT ...;
```

সংক্ষেপে: overwrite করলে history **পরিষ্কার হয় না, বরং পুরোটা থেকে যায়**। তাই ভুল যুক্তি হলো **B**।

Question 12

QUESTION

Which of the following commands will return records from an existing Delta table my_table where duplicates have been removed?

- A. DROP DUPLICATES FROM my_table;
- B. SELECT * FROM my_table WHERE duplicate = False;
- C. SELECT DISTINCT * FROM my_table;
- D. MERGE INTO my_table a USING new_records b ON a.id = b.id WHEN NOT MATCHED THEN INSERT *;
- E. MERGE INTO my_table a USING new_records b;

✓ সঠিক উত্তর: C

```
SELECT DISTINCT * FROM my_table;
```

ব্যাখ্যা

`SELECT DISTINCT` ফলাফল থেকে **ভ্রূত একই রকম row** গুলো বাদ দিয়ে প্রতিটা আলাদা row মাত্র **একবার** দেখায়। `*` মানে সব column মিলিয়ে তুলনা করা হবে। অর্থাৎ দুটো row-এর **সব column-এর মান এক হলে** তবেই সেগুলো duplicate ধরা হয়।

উদাহরণ: my_table :

id	name
1	Rahim
2	Karim
1	Rahim

`SELECT DISTINCT * FROM my_table;` চললে:

id	name
1	Rahim
2	Karim

💡 খেয়াল করুন: `SELECT DISTINCT` শুধু ফলাফল **দেখায় (return করে)**, মূল table থেকে duplicate **মুছে ফেলে না**। প্রশ্নেও শুধু record ফেরত দিতে বলা হয়েছে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	SQL-এ DROP DUPLICATES বলে কোনো command নেই। (PySpark DataFrame-এ <code>dropDuplicates()</code> method আছে, কিন্তু সেটা SQL command নয়।)
B	Table-এ duplicate নামে কোনো column নেই। Delta Lake নিজে থেকে এমন কোনো column বানায় না। এটা error দেবে।
D	এটা একটা MERGE (upsert) operation। <code>new_records</code> থেকে শুধু সেই record গুলো <code>my_table</code> -এ insert করে যেগুলো আগে থেকে নেই। এতে নতুন duplicate ঢোকা আটকায়, কিন্তু এটা table-এ লেখে , record ফেরত দেয় না । আর <code>my_table</code> -এ আগে থেকে থাকা duplicate-ও সরায় না।
E	ON শর্ত আর WHEN MATCHED / NOT MATCHED অংশ নেই, তাই অসম্পূর্ণ syntax । এটা error দেবে।

🔑 মনে রাখার উপায়

কাজ	উপায়
Duplicate বাদ দিয়ে দেখানো (SQL)	<code>SELECT DISTINCT * FROM t</code> ✅
Duplicate বাদ দিয়ে দেখানো (PySpark)	<code>df.dropDuplicates()</code> / <code>df.distinct()</code>
নির্দিষ্ট column দেখে duplicate বাদ (PySpark)	<code>df.dropDuplicates(["id"])</code>
Insert করার সময় duplicate ঢোকা আটকানো	<code>MERGE INTO ... WHEN NOT MATCHED THEN INSERT *</code>
Table থেকে duplicate স্থায়ীভাবে সরানো	<code>CREATE OR REPLACE TABLE t AS SELECT DISTINCT * FROM t</code>

সংক্ষেপে: duplicate বাদ দিয়ে record ফেরত দেয় `SELECT DISTINCT *`। তাই উত্তর **C**।

Question 13

QUESTION

A data engineer wants to horizontally combine two tables as a part of a query. They want to use a shared column as a key column, and they only want the query result to contain rows whose value in the key column is present in both tables.

Which of the following SQL commands can they use to accomplish this task?

- A. INNER JOIN
- B. OUTER JOIN
- C. LEFT JOIN
- D. MERGE
- E. UNION

✓ সঠিক উত্তর: A. INNER JOIN

ব্যাখ্যা

প্রশ্নে তিনটি শর্ত আছে:

1. **Horizontally combine** → দুই table-এর column পাশাপাশি জোড়া লাগানো। এটা **JOIN**-এর কাজ।
2. **Shared column-কে key হিসেবে ব্যবহার** → `ON a.key = b.key`
3. **শুধু দুই table-এই থাকা মানগুলো** → এটাই **INNER JOIN**-এর কাজ। দুই table-এ মিল আছে এমন row-ই শুধু ফলাফলে আসে।

```
SELECT *  
FROM customers c  
INNER JOIN orders o  
ON c.customer_id = o.customer_id;
```

উদাহরণ:

customers :

customer_id	name
1	Rahim
2	Karim
3	Salma

orders :

customer_id	amount
1	500
3	300
4	700

INNER JOIN-এর ফলাফল (শুধু 1 আর 3, কারণ এই দুটো মানই দুই table-এ আছে):

customer_id	name	amount
1	Rahim	500
3	Salma	300

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B. OUTER JOIN	(FULL OUTER JOIN) দুই table-এর সব row আনে। মিল না থাকলে খালি জায়গায় NULL বসায়। উদাহরণে 1, 2, 3, 4 সবই আসত।
C. LEFT JOIN	বাম table-এর সব row আনে, ডান table-এ মিল থাকুক বা না থাকুক। উদাহরণে 1, 2, 3 আসত (2-এর amount NULL)।
D. MERGE	এটা query-তে table মেলানোর জন্য নয়। এটা একটা table-এ insert/update/delete (upsert) করার command, অর্থাৎ table-এ লেখে।
E. UNION	দুটো table-কে উপর-নিচে (vertically) জোড়া লাগায়, অর্থাৎ row যোগ করে। পাশাপাশি column জোড়া লাগায় না, আর key column-ও ব্যবহার করে না।

🔑 মনে রাখার উপায়

JOIN ধরন	কোন row আসে
INNER JOIN	শুধু দুই table-এই মিল আছে এমন row ✅
LEFT JOIN	বাম table-এর সব + ডান থেকে মিলগুলো
RIGHT JOIN	ডান table-এর সব + বাম থেকে মিলগুলো
FULL OUTER JOIN	দুই table-এর সব row
LEFT SEMI JOIN	বাম table-এর যে row-এর মিল আছে (শুধু বাম table-এর column)
LEFT ANTI JOIN	বাম table-এর যে row-এর মিল নেই

💡 **JOIN = পাশাপাশি (horizontal), UNION = উপর-নিচে (vertical)**

সংক্ষেপে: শুধু দুই table-এই থাকা key মানের row আনে **INNER JOIN**। তাই উত্তর **A**।

Question 14

QUESTION

A junior data engineer has ingested a JSON file into a table `raw_table` with the following schema:

```
cart_id STRING,  
items ARRAY<item_id:STRING>
```

The junior data engineer would like to unnest the `items` column in `raw_table` to result in a new table with the following schema:

```
cart_id STRING,  
item_id STRING
```

Which of the following commands should the junior data engineer run to complete this task?

- A. `SELECT cart_id, filter(items) AS item_id FROM raw_table;`
- B. `SELECT cart_id, flatten(items) AS item_id FROM raw_table;`
- C. `SELECT cart_id, reduce(items) AS item_id FROM raw_table;`
- D. `SELECT cart_id, explode(items) AS item_id FROM raw_table;`
- E. `SELECT cart_id, slice(items) AS item_id FROM raw_table;`

✓ সঠিক উত্তর: D. `explode()`

```
SELECT cart_id, explode(items) AS item_id FROM raw_table;
```

ব্যাখ্যা

`explode()` একটি array-র প্রতিটি element-কে আলাদা row-তে ভেঙে দেয়। Row-এর বাকি column (এখানে `cart_id`) প্রতিটি নতুন row-তে আবার বসে যায়। একেই **unnest** বলে।

উদাহরণ: `raw_table`:

cart_id	items
c1	[i1, i2, i3]
c2	[i4]

`explode(items)` -এর পরে:

cart_id	item_id
c1	i1
c1	i2
c1	i3
c2	i4

এক row-এ একটা array থেকে প্রতিটা item-এর জন্য একটা করে row হয়ে গেল, ঠিক যে schema চাওয়া হয়েছিল।

💡 বাস্তবে array-র element গুলো যদি struct হয় (যেমন `ARRAY<STRUCT<item_id: STRING>>`), তাহলে explode করার পর `.item_id` দিয়ে field টা বের করতে হয়। পরীক্ষার জন্য মূল কথা হলো: **array → row = explode**।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. filter()	Array-র ভেতর থেকে শর্ত অনুযায়ী element বাছাই করে , আর ফলাফল আবার একটা array-ই থাকে। Row তৈরি করে না। এর জন্য একটা lambda function-ও লাগে: <code>filter(items, x -> x > 5)</code>
B. flatten()	Array-র ভেতরে থাকা array (nested array) কে একটা array-তে পরিণত করে। যেমন <code>[[1,2],[3]] → [1,2,3]</code> । ফলাফল তখনও array, আলাদা row নয়।
C. reduce()	Array-র সব element মিলিয়ে একটা মাত্র মান বানায় (যেমন যোগফল)। Row ভাঙে না। (এর জন্য initial value আর lambda-ও লাগে।)
E. slice()	Array-র একটা অংশ কেটে নেয় । যেমন <code>slice(arr, 1, 2)</code> প্রথম ২টা element দেয়। ফলাফল আবার array।

🔑 মনে রাখার উপায়

Function	কাজ	ফলাফল
<code>explode()</code>	Array → আলাদা row ✅	একাধিক row
<code>flatten()</code>	Nested array → একটা array	Array
<code>filter()</code>	শর্ত মেনে element বাছাই	Array
<code>reduce()</code> / <code>aggregate()</code>	সব element মিলিয়ে একটা মান	একটা মান
<code>slice()</code>	Array-র একটা অংশ	Array
<code>collect_list()</code>	Row → array (explode-এর উল্টো)	Array

💡 **explode = array ভেঙে row বানানো, collect_list = row জুড়ে array বানানো**

সংক্ষেপে: array column-কে আলাদা row-তে unnest করে `explode()`। তাই উত্তর **D**।

Question 15

QUESTION

A data engineer has ingested a JSON file into a table `raw_table` with the following schema:

```
transaction_id STRING,  
payload ARRAY<customer_id:STRING, date:TIMESTAMP, store_id:STRING>
```

The data engineer wants to efficiently extract the date of each transaction into a table with the following schema:

```
transaction_id STRING,  
date TIMESTAMP
```

Which of the following commands should the data engineer run to complete this task?

- A. `SELECT transaction_id, explode(payload) FROM raw_table;`
- B. `SELECT transaction_id, payload.date FROM raw_table;`
- C. `SELECT transaction_id, date FROM raw_table;`
- D. `SELECT transaction_id, payload[date] FROM raw_table;`
- E. `SELECT transaction_id, date from payload FROM raw_table;`

✓ সঠিক উত্তর: B

```
SELECT transaction_id, payload.date FROM raw_table;
```

ব্যাখ্যা

`payload` column-এর ভেতরে কয়েকটা **sub-field** আছে: `customer_id`, `date`, `store_id`। Nested (struct) data থেকে নির্দিষ্ট একটা field বের করতে Spark SQL-এ **dot notation** (`.`) ব্যবহার করা হয়:

```
column_name.field_name → payload.date
```

এতে শুধু দরকারি field টা (`date`) সরাসরি বের হয়ে আসে। পুরো structure ভাঙতে হয় না, তাই এটাই সবচেয়ে **সহজ ও দক্ষ (efficient)** উপায়।

উদাহরণ: `raw_table` :

transaction_id	payload
t1	{customer_id: c1, date: 2024-01-05 10:00, store_id: s1}
t2	{customer_id: c2, date: 2024-02-10 14:30, store_id: s2}

`SELECT transaction_id, payload.date` -এর ফলাফল:

transaction_id	date
t1	2024-01-05 10:00
t2	2024-02-10 14:30

💡 প্রস্নে `payload` -কে `ARRAY` লেখা হলেও, এর গঠন আসলে **struct**-এর মতো (নামসহ field আছে)। পরীক্ষার জন্য মূল কথা হলো: **nested field বের করতে dot notation**। (বাস্তবে এটা struct-এর array হলে `payload.date` `dates`-এর একটা array ফেরত দিত।)

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	<code>explode()</code> array-কে আলাদা row-তে ভাঙে, কিন্তু পুরো struct টা ফেরত দেয় (<code>customer_id</code> , <code>date</code> , <code>store_id</code> সব)। শুধু <code>date</code> আলাদা হয় না, ফলে চাওয়া schema-ও মেলে না। এছাড়া এটা বাড়তি কাজ, তাই কম দক্ষ।
C	<code>date</code> নামে top-level কোনো column নেই । <code>date</code> আছে <code>payload</code> -এর ভেতরে। তাই এটা error দেবে।
D	Bracket <code>[]</code> ব্যবহার হয় array-র index (যেমন <code>payload[0]</code>) বা map-এর key (যেমন <code>map['key']</code>) দিয়ে মান বের করতে। Struct field-এর জন্য <code>[date]</code> ভুল syntax।
E	<code>SELECT ... from payload FROM raw_table</code> , এখানে দুইবার FROM । এটা পুরোপুরি ভুল syntax ।

🔑 মনে রাখার উপায়

Data type	মান বের করার উপায়	উদাহরণ
Struct	Dot notation <code>.</code> ✅	<code>payload.date</code>
Array	Index <code>[n]</code> (0 থেকে শুরু)	<code>items[0]</code>
Map	Key <code>['key']</code>	<code>props['color']</code>
Array → row	<code>explode()</code>	<code>explode(items)</code>
JSON string	<code>:</code> (colon) syntax	<code>json_col:customer.name</code>

সংক্ষেপে: nested field থেকে `date` সরাসরি বের করার সবচেয়ে দক্ষ উপায় হলো **dot notation (`payload.date`)**। তাই উত্তর **B**।

Question 16

QUESTION

A data analyst has provided a data engineering team with the following Spark SQL query:

```
SELECT district,  
        avg(sales)  
FROM store_sales_20220101  
GROUP BY district;
```

The data analyst would like the data engineering team to run this query every day. The date at the end of the table name (20220101) should automatically be replaced with the current date each time the query is run.

Which of the following approaches could be used by the data engineering team to efficiently automate this process?

- A. They could wrap the query using PySpark and use Python's string variable system to automatically update the table name.
- B. They could manually replace the date within the table name with the current day's date.
- C. They could request that the data analyst rewrites the query to be run less frequently.
- D. They could replace the string-formatted date in the table with a timestamp-formatted date.
- E. They could pass the table into PySpark and develop a robustly tested module on the existing query.

✓ সঠিক উত্তর: A

ব্যাখ্যা

SQL-এ table-এর নাম সাধারণত **fixed** থাকে, সেটা নিজে থেকে বদলানো যায় না। কিন্তু **Python দিয়ে query টা একটা string হিসেবে বানাতে**, তারিখের জায়গায় একটা variable বসানো যায়। তারপর `spark.sql()` দিয়ে query টা চালানো যায়।

```
from datetime import datetime

# আজকের তারিখ YYYYMMDD format-এ
today = datetime.now().strftime("%Y%m%d")

# f-string দিয়ে table-এর নামে তারিখ বসানো
query = f"""
SELECT district,
       avg(sales)
FROM store_sales_{today}
GROUP BY district
"""

result = spark.sql(query)
```

প্রতিদিন চালালে `today` variable-এ সেদিনের তারিখ আসবে (যেমন `20240315`), আর table-এর নাম হয়ে যাবে `store_sales_20240315`। এরপর এই notebook টা একটা **Databricks Job** হিসেবে প্রতিদিন চালানোর জন্য schedule করে দিলেই পুরো কাজটা automate হয়ে যায়।

এটা **সহজ, কম কাজের, আর দক্ষ** সমাধান।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	প্রতিদিন হাতে বদলানো মানে automation না। এতে ভুল হওয়ার সম্ভাবনা থাকে, আর সময়ও নষ্ট হয়।
C	এটা সমস্যার সমাধান নয়। Analyst-এর দরকার প্রতিদিনের ফলাফল, কম ঘন ঘন চালালে সেই চাহিদা মেটে না।
D	Table-এর নামে timestamp format বসালেও নামটা নিজে থেকে বদলাবে না। তাছাড়া table-এর নামে <code>:</code> বা space-এর মতো অক্ষর বসানো যায় না। এটা কোনো automation নয় ।
E	একটা ছোট query-র জন্য পুরো module বানানো আর test করা অপ্রয়োজনীয়ভাবে জটিল । প্রশ্নে "efficiently" বলা হয়েছে, আর option A-ই সবচেয়ে সহজ সমাধান।

🔑 মনে রাখার উপায়

প্রয়োজন	সমাধান
SQL query-তে dynamic মান (table নাম, তারিখ)	Python f-string + <code>spark.sql()</code> ✅
Notebook প্রতিদিন চালানো	Databricks Jobs দিয়ে schedule
Notebook-এ বাইরে থেকে মান পাঠানো	Widgets / Job parameters

💡 **SQL = fixed, Python = dynamic**। দুটো একসাথে ব্যবহার করলে (`spark.sql(f"...")`) query-কে flexible করা যায়।

সংক্ষেপে: Python-এর **string variable (f-string)** দিয়ে table-এর নামে আজকের তারিখ বসিয়ে query চালানোই সবচেয়ে দক্ষ সমাধান। তাই উত্তর **A**।

Question 17

QUESTION

A data engineer has ingested data from an external source into a PySpark DataFrame `raw_df`. They need to briefly make this data available in SQL for a data analyst to perform a quality assurance check on the data.

Which of the following commands should the data engineer run to make this data available in SQL for only the remainder of the Spark session?

- A. `raw_df.createOrReplaceTempView("raw_df")`
- B. `raw_df.createTable("raw_df")`
- C. `raw_df.write.save("raw_df")`
- D. `raw_df.saveAsTable("raw_df")`
- E. There is no way to share data between PySpark and SQL.

✓ সঠিক উত্তর: A

```
raw_df.createOrReplaceTempView("raw_df")
```

ব্যাখ্যা

`createOrReplaceTempView()` একটা DataFrame থেকে **temporary view** বানায়। এর বৈশিষ্ট্যগুলো প্রশ্নের শর্তের সাথে পুরোপুরি মেলে:

1. **SQL-এ ব্যবহার করা যায়** → view বানানোর পর সরাসরি SQL দিয়ে query চালানো যায়
2. **শুধু session-এর বাকি সময়ের জন্য** → temporary view **session শেষ হলে নিজে থেকেই মুছে যায়**
3. **কোনো data disk-এ লেখা হয় না** → এটা শুধু DataFrame-এর একটা নাম, তাই storage খরচ নেই
4. **"OrReplace"** → একই নামের view আগে থেকে থাকলে সেটা replace করে দেয়, error দেয় না

```
# PySpark-এ temp view বানানো
raw_df.createOrReplaceTempView("raw_df")
```

```
-- এখন SQL-এ query চালানো যায়
SELECT * FROM raw_df WHERE customer_id IS NULL;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	DataFrame-এ <code>createTable()</code> বলে কোনো method নেই। এটা error দেবে।
C	<code>write.save()</code> data-কে file হিসেবে disk-এ লিখে রাখে (স্থায়ীভাবে)। এতে SQL table বা view তৈরি হয় না, আর session শেষ হলেও data থেকে যায়।
D	<code>saveAsTable()</code> একটা স্থায়ী (persistent) table বানায়, যা session শেষ হলেও থেকে যায়। তাছাড়া সঠিক syntax হলো <code>raw_df.write.saveAsTable(...)</code> , সরাসরি <code>raw_df.saveAsTable(...)</code> নয়।
E	ভুল। Temp view, global temp view বা table দিয়ে PySpark আর SQL-এর মধ্যে সহজেই data share করা যায়। আবার উল্টো দিকে <code>spark.sql()</code> বা <code>spark.table()</code> দিয়ে SQL থেকে DataFrame-এ আনা যায়।

🔑 মনে রাখার উপায়

Command	কতক্ষণ থাকে	কোথা থেকে পাওয়া যায়
<code>createOrReplaceTempView()</code>	Session শেষ হওয়া পর্যন্ত ✅	শুধু নিজের session
<code>createOrReplaceGlobalTempView()</code>	Cluster চালু থাকা পর্যন্ত	একই cluster-এর সব session (<code>global_temp.view_name</code> দিয়ে)
<code>write.saveAsTable()</code>	স্থায়ী	সবাই (metastore-এ save থাকে)
<code>write.save()</code>	স্থায়ী (file হিসেবে)	Path দিয়ে

💡 PySpark → SQL: `df.createOrReplaceTempView("name")` 💡 SQL → PySpark: `spark.sql("SELECT ...")` অথবা `spark.table("name")`

সংক্ষেপে: শুধু session-এর বাকি সময়ের জন্য DataFrame-কে SQL-এ পাওয়ার ব্যবস্থা করে `createOrReplaceTempView()` । তাই উত্তর A।

Question 18

QUESTION

🚫 **OUT OF CURRENT SYLLABUS** – এই প্রশ্নটি বর্তমান syllabus (May 2026 exam guide)-এর বাইরে

কারণ: Pure Python language basics (PySpark নয়) – কোনো exam objective-এ নেই

A data engineer needs to dynamically create a table name string using three Python variables: region, store, and year. An example of a table name is below when region = "nyc", store = "100", and year = "2021":

```
nyc100_sales_2021
```

Which of the following commands should the data engineer use to construct the table name in Python?

- A. "{region}+{store}+_sales_{year}"
- B. f"{region}+{store}+_sales_{year}"
- C. "{region}{store}_sales_{year}"
- D. f"{region}{store}_sales_{year}"
- E. {region}+{store}+"_sales_" + {year}

✅ সঠিক উত্তর: D

```
region = "nyc"
store = "100"
year = "2021"

table_name = f"{region}{store}_sales_{year}"
print(table_name) # nyc100_sales_2021
```

ব্যাখ্যা

f-string (formatted string literal) Python-এ string-এর ভেতরে variable বসানোর সবচেয়ে সহজ উপায়।

- String-এর শুরুতে **f** লিখতে হয়
- Variable-এর নাম **{ }** -এর ভেতরে লিখলে সেখানে variable-এর মান বসে যায়
- বাকি সব লেখা (যেমন `_sales_`) **হুবহু** থাকে

তাই `f"{region}{store}_sales_{year}"` হয়ে যায়:

```
{region} → nyc
{store} → 100
_sales_ → _sales_
{year} → 2021
-----
nyc100_sales_2021 ✓
```

✗ বাকি option গুলো কেন ভুল

Option	ফলাফল	কেন ভুল
A	<code>{region}+{store}+_sales_+{year}</code>	শুরুতে f নেই, তাই <code>{ }</code> -এর ভেতরের variable বদলায় না, হুবহু লেখা হিসেবে থেকে যায়। <code>+</code> চিহ্নও string-এর অংশ হয়ে যায়।
B	<code>nyc+100+_sales_+2021</code>	<code>f</code> আছে বলে variable বসে যায়, কিন্তু f-string-এর ভেতরে + সাধারণ লেখা (literal) হিসেবে থেকে যায়। ফলে নামে <code>+</code> চিহ্ন চলে আসে।
C	<code>{region} {store}_sales_{year}</code>	f নেই, তাই variable-এর মান বসে না।
E	✗ Error	Python-এ <code>{region}</code> লিখলে সেটা variable নয়, একটা set তৈরি হয়। আর set-কে <code>+</code> দিয়ে string-এর সাথে যোগ করা যায় না, তাই TypeError দেবে।

🔑 মনে রাখার উপায়

উপায়	উদাহরণ	ফলাফল
f-string ✓	<code>f"{region}{store}_sales_{year}"</code>	<code>nyc100_sales_2021</code>
<code>+</code> দিয়ে জোড়া	<code>region + store + "_sales_" + year</code>	<code>nyc100_sales_2021</code>
<code>.format()</code>	<code>"{}{}_sales{}".format(region, store, year)</code>	<code>nyc100_sales_2021</code>
<code>f</code> ছাড়া	<code>"{region}{store}_sales_{year}"</code>	<code>{region}{store}_sales_{year}</code> ✗

💡 **f-string-এর নিয়ম:** শুরুতে `f`, variable থাকবে `{ }`-এর ভেতরে, আর ভেতরে `+` লাগে না।

সংক্ষেপে: f-string দিয়ে variable সরাসরি string-এর ভেতরে বসাতে হয়, কোনো `+` ছাড়া। তাই উত্তর **D**।

Question 19

QUESTION

A data engineer has developed a code block to perform a streaming read on a data source. The code block is below:

```
(spark
  .read
  .schema(schema)
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .load(dataSource)
)
```

The code block is returning an error.

Which of the following changes should be made to the code block to configure the block to successfully perform a streaming read?

- A. The `.read` line should be replaced with `.readStream`.
- B. A new `.stream` line should be added after the `.read` line.
- C. The `.format("cloudFiles")` line should be replaced with `.format("stream")`.
- D. A new `.stream` line should be added after the `spark` line.
- E. A new `.stream` line should be added after the `.load(dataSource)` line.

✓ সঠিক উত্তর: A

```
(spark
  .readStream                                # ✓ .read → .readStream
  .schema(schema)
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .load(dataSource)
)
```

ব্যাখ্যা

সমস্যাটা কোথায়? `format("cloudFiles")` মানে **Auto Loader**। Auto Loader হলো Databricks-এর একটা সুবিধা যা cloud storage-এ নতুন আসা file গুলো নিজে থেকে খুঁজে নিয়ে ধাপে ধাপে (incrementally) load করে। Auto Loader শুধু **Structured Streaming** দিয়েই কাজ করে।

কিন্তু code-এ লেখা হয়েছে `spark.read`, যা একটা **batch read**। Batch read দিয়ে `cloudFiles` format চালানো যায় না, তাই error আসছে।

সমাধান: `spark.read` -এর জায়গায় `spark.readStream` লিখতে হবে। `readStream` একটা streaming DataFrame বানায়, যা Auto Loader-এর সাথে কাজ করে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	Spark-এ <code>.stream</code> বলে কোনো method নেই। <code>.read</code> -এর পরে এটা যোগ করলে error আসবে।
C	<code>"stream"</code> কোনো বৈধ format নয়। Auto Loader-এর format হলো <code>"cloudFiles"</code> , এটা ঠিকই আছে। সমস্যা <code>.read</code> -এ।
D	একই কারণে ভুল। <code>.stream</code> method-এর অস্তিত্ব নেই।
E	<code>.load()</code> -এর পরে <code>.stream</code> যোগ করলেও কাজ হবে না, কারণ এমন কোনো method নেই।

🔑 মনে রাখার উপায়

কাজ	Read	Write
Batch	<code>spark.read</code>	<code>df.write</code>
Streaming	<code>spark.readStream</code> ✅	<code>df.writeStream</code>

সম্পূর্ণ Auto Loader উদাহরণ (read + write):

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", schema_path)
  .load(dataSource)
  .writeStream
    .option("checkpointLocation", checkpoint_path)
    .table("target_table")
)
```

💡 `format("cloudFiles")` = Auto Loader = সবসময় `readStream`

সংক্ষেপে: Auto Loader (`cloudFiles`) শুধু streaming-এ চলে, তাই `.read` -এর বদলে `.readStream` লিখতে হবে। উত্তর A।

Question 20

QUESTION

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  ._____
  .table("new_sales")
)
```

If the data engineer only wants the query to execute a single micro-batch to process all of the available data, which of the following lines of code should the data engineer use to fill in the blank?

- A. `trigger(once=True)`
- B. `trigger(continuous="once")`
- C. `processingTime("once")`
- D. `trigger(processingTime="once")`
- E. `processingTime(1)`

✓ সঠিক উত্তর: A. `trigger(once=True)`

ব্যাখ্যা

Structured Streaming-এ **trigger** ঠিক করে দেয় query **কখন আর কতবার** data process করবে।

`trigger(once=True)` দিলে:

1. Query চালু হয়
2. সেই মুহূর্তে যত নতুন data আছে, সব **একটা মাত্র micro-batch-এ** process করে
3. তারপর **নিজে থেকেই বন্ধ হয়ে যায়**

এটা streaming-এর সুবিধা (checkpoint দিয়ে শুধু নতুন data process করা) আর batch-এর সুবিধা (চালিয়ে বন্ধ করা যায়, cluster সারাক্ষণ চালু রাখতে হয় না) দুটোই দেয়। দিনে একবার বা ঘণ্টায় একবার schedule করা job-এর জন্য এটা খুব কাজের।

💡 নতুন Databricks version-এ এর বদলে `trigger(availableNow=True)` ব্যবহার করার পরামর্শ দেওয়া হয়। এটাও সব available data process করে বন্ধ হয়, তবে প্রয়োজনে **একাধিক micro-batch-এ** ভাগ করে চালায়, যা বড় data-র জন্য ভালো। কিন্তু এই প্রশ্নে **"single micro-batch"** চাওয়া হয়েছে, তাই উত্তর `once=True`।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	<code>continuous</code> trigger-এ একটা সময়ের interval দিতে হয় (যেমন <code>continuous="1 second"</code>)। এটা খুব কম latency-র continuous processing mode , একবার চালানোর জন্য নয়। <code>"once"</code> এখানে বৈধ মান নয়।
C	<code>processingTime</code> আলাদা কোনো method নয়, এটা <code>trigger()</code> -এর একটা parameter । তাই <code>.processingTime(...)</code> লিখলে error আসবে।
D	<code>processingTime</code> -এ দিতে হয় সময়ের interval (যেমন <code>"5 seconds"</code> , <code>"10 minutes"</code>)। <code>"once"</code> কোনো সময় নয়, তাই এটা ভুল।
E	একই সমস্যা: <code>processingTime</code> আলাদা method নয়, আর এখানে <code>trigger()</code> -ও নেই।

🔑 মনে রাখার উপায়

Trigger	কাজ
(কিছু না দিলে, default)	আগের micro-batch শেষ হলেই পরেরটা শুরু, সারাক্ষণ চলে
<code>trigger(processingTime="5 minutes")</code>	প্রতি ৫ মিনিট পর পর একটা micro-batch, সারাক্ষণ চলে
<code>trigger(once=True)</code>	একটা মাত্র micro-batch , তারপর বন্ধ ✅
<code>trigger(availableNow=True)</code>	সব available data (একাধিক micro-batch হতে পারে), তারপর বন্ধ
<code>trigger(continuous="1 second")</code>	Continuous processing (খুব কম latency)

💡 "single micro-batch" → `once=True` 💡 "all available data, multiple batches" → `availableNow=True`

ছোট একটা বিষয়: streaming পড়ার জন্য সাধারণত `spark.readStream.table("sales")` লেখা হয়। প্রশ্নে `spark.table()` আছে, তবে পরীক্ষার জন্য মূল বিষয় হলো খালি জায়গার **trigger**।
সংক্ষেপে: একটা মাত্র micro-batch-এ সব data process করে বন্ধ হতে হলে লাগে `trigger(once=True)`। তাই উত্তর **A**।

Question 21

QUESTION

A data engineer is designing a data pipeline. The source system generates files in a shared directory that is also used by other processes. As a result, the files should be kept as is and will accumulate in the directory. The data engineer needs to identify which files are new since the previous run in the pipeline, and set up the pipeline to only ingest those new files with each run.

Which of the following tools can the data engineer use to solve this problem?

- A. Databricks SQL
- B. Delta Lake
- C. Unity Catalog
- D. **Catalog Explorer**
- E. Auto Loader

✓ সঠিক উত্তর: E. Auto Loader

ব্যাখ্যা

Auto Loader ঠিক এই কাজের জন্যই বানানো। Cloud storage-এর একটি directory-তে **নতুন আসা file গুলো নিজে থেকে চিনে নিয়ে শুধু সেগুলোকেই ধাপে ধাপে (incrementally) load করে।**

Auto Loader কীভাবে নতুন file চেনে?

- কোন কোন file আগেই process হয়েছে, তার রেকর্ড Auto Loader **checkpoint location**-এ রাখে (RocksDB-তে)
- প্রতিবার চালানোর সময় এই রেকর্ডের সাথে মিলিয়ে দেখে, **শুধু নতুন file গুলো** process করে
- Source directory-র file **সরায় না, মোছে না, বদলায় না।** তাই অন্য process-গুলোর কোনো সমস্যা হয় না
- প্রতিটা file **ঠিক একবারই** process হয় (exactly-once)

প্রশ্নের সব শর্তই মেলে:

শর্ত	Auto Loader
File যেমন আছে তেমনই থাকবে	✓ Source file-এ হাত দেয় না
File জমতে থাকবে	✓ লাখো file থাকলেও দক্ষভাবে কাজ করে
আগের run-এর পর নতুন file চেনা	✓ Checkpoint-এ রেকর্ড রাখে
শুধু নতুন file ingest করা	✓ Incremental ingestion

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", schema_path)
  .load("/shared/source_directory")
  .writeStream
  .option("checkpointLocation", checkpoint_path)
  .trigger(availableNow=True)      # প্রতি run-এ নতুন file process করে বন্ধ হয়
  .table("bronze_table")
)
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. Databricks SQL	SQL query চালানো, dashboard আর visualization-এর জন্য। নতুন file চিনে নিজে থেকে ingest করার সুবিধা এতে নেই।
B. Delta Lake	Data সংরক্ষণের storage layer (ACID, time travel ইত্যাদি)। Ingest হওয়ার পরে data রাখার জায়গা, কিন্তু source directory-তে নতুন file খোঁজার কাজ এটা করে না।
C. Unity Catalog	Data governance tool, অর্থাৎ access control, permission, lineage, auditing। File ingestion-এর সাথে এর সম্পর্ক নেই।
D. Data Explorer	Database, table আর permission দেখা ও সামলানোর UI । এটা কোনো ingestion tool নয়।

🔑 মনে রাখার উপায়

Tool	মূল কাজ
Auto Loader	Cloud storage থেকে নতুন file incrementally ingest ✅
COPY INTO	SQL দিয়ে file load (এটাও idempotent, আগে load হওয়া file বাদ দেয়; হাজারখানেক file পর্যন্ত ভালো)
Delta Lake	Data storage layer
Unity Catalog	Governance ও security
Databricks SQL	Query, dashboard, BI
Data Explorer	Data/permission দেখার UI

💡 "শুধু নতুন file" + "cloud storage directory" → **Auto Loader** 💡 Auto Loader =
format("cloudFiles") + checkpoint

সংক্ষেপে: shared directory-র file না ছুঁয়ে শুধু নতুন file গুলো চিনে প্রতিবার ingest করে **Auto Loader**। তাই উত্তর **E**।

Question 22

QUESTION

A data engineering team is in the process of converting their existing data pipeline to utilize Auto Loader for incremental processing in the ingestion of JSON files. One data engineer comes across the following code block in the Auto Loader documentation:

```
(streaming_df = spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", schemaLocation)
  .load(sourcePath))
```

Assuming that schemaLocation and sourcePath have been set correctly, which of the following changes does the data engineer need to make to convert this code block to use Auto Loader to ingest the data?

- A. The data engineer needs to change the format("cloudFiles") line to format("autoLoader").
- B. There is no change required. Databricks automatically uses Auto Loader for streaming reads.
- C. There is no change required. The inclusion of format("cloudFiles") enables the use of Auto Loader.
- D. The data engineer needs to add the .autoLoader line before the .load(sourcePath) line.
- E. There is no change required. The data engineer needs to ask their administrator to turn on Auto Loader.

✓ সঠিক উত্তর: C

ব্যাখ্যা

Databricks-এ **Auto Loader চালু করার উপায় হলো** `format("cloudFiles")` । আলাদা কোনো method, keyword বা setting লাগে না।

Code block-টা লাইন ধরে দেখা যাক:

Line	কাজ
<code>spark.readStream</code>	Streaming read শুরু (Auto Loader শুধু streaming-এ চলে)
<code>.format("cloudFiles")</code>	Auto Loader চালু করে ✓
<code>.option("cloudFiles.format", "json")</code>	Source file গুলো কোন format-এর (এখানে JSON)
<code>.option("cloudFiles.schemaLocation", ...)</code>	Detect করা schema কোথায় save হবে (schema inference ও evolution-এর জন্য)
<code>.load(sourcePath)</code>	কোন directory থেকে file পড়তে হবে

তাই code টা **আগে থেকেই Auto Loader ব্যবহার করছে**, কোনো পরিবর্তনের দরকার নেই।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	"autoLoader" কোনো বৈধ format নয়। Auto Loader-এর format-এর নাম হলো "cloudFiles"।
B	Databricks streaming read-এ নিজে থেকে Auto Loader ব্যবহার করে না। যেমন format("json") দিয়ে readStream করলে সাধারণ file streaming source চলে, Auto Loader নয়। Auto Loader পেতে হলে cloudFiles বলে দিতেই হবে।
D	Spark-এ .autoLoader বলে কোনো method নেই। এটা error দেবে।
E	Auto Loader Databricks-এ আগে থেকেই পাওয়া যায়। Admin-কে দিয়ে আলাদা করে চালু করাতে হয় না।

🔑 মনে রাখার উপায়

Code	কী হয়
spark.readStream.format("cloudFiles")	✅ Auto Loader
spark.readStream.format("json")	সাধারণ file streaming (Auto Loader নয়)
spark.read.format("cloudFiles")	❌ Error (Auto Loader-এ readStream লাগে)

Auto Loader-এর মূল option গুলো:

Option	কাজ
cloudFiles.format	Source file-এর format (json, csv, parquet ইত্যাদি)
cloudFiles.schemaLocation	Schema save রাখার জায়গা
cloudFiles.inferColumnTypes	Column-এর type অনুমান করা (না দিলে JSON/CSV-তে সব STRING হয়)
cloudFiles.schemaEvolutionMode	নতুন column এলে কী করবে

💡 **Auto Loader = readStream + format("cloudFiles")**। এর বেশি কিছু লাগে না।

সংক্ষেপে: format("cloudFiles") থাকতেই Auto Loader চালু হয়ে যায়, তাই কোনো পরিবর্তনের দরকার নেই। উত্তর C।

Question 23

QUESTION

Which of the following data workloads will utilize a Bronze table as its source?

- A. A job that aggregates cleaned data to create standard summary statistics
- B. A job that queries aggregated data to publish key insights into a dashboard
- C. A job that ingests raw data from a streaming source into the Lakehouse
- D. A job that develops a feature set for a machine learning application
- E. A job that enriches data by parsing its timestamps into a human-readable format

✓ সঠিক উত্তর: E

ब्यांथरा

Databricks-এর **Medallion Architecture**-এ data তিনটি ধাপে (layer-এ) সাজানো হয়:



Layer	কী থাকে	কাজ
 Bronze	Raw data, যেমন এসেছে প্রায় তেমনই	Source থেকে ingest
 Silver	পরিষ্কার, যাচাই করা, enrich করা data	Cleaning, filtering, parsing, join, deduplicate
 Gold	Aggregate করা, business-ready data	Summary, report, dashboard, ML

প্রশ্নে জানতে চাওয়া হয়েছে কোন job **Bronze থেকে data পড়ে**। Bronze থেকে data পড়ে **Silver বানানো** হয়, আর Silver-এর কাজ হলো data পরিষ্কার ও enrich করা।

Option **E**-তে timestamp parse করে পড়ার মতো format-এ আনা (data enrich করা) হলো ঠিক **Bronze** → **Silver** ধাপের কাজ:

```
from pyspark.sql.functions import col, from_unixtime

(spark.readStream.table("bronze_events")
    .withColumn("event_time",
                from_unixtime(col("timestamp")))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .table("silver_events"))
```

🏅 Bronze থেকে পড়া

timestamp → পড়ার মতো format

🥈 Silver-এ লেখা

❌ বাকি option গুলো কেন ভুল

Option	Source কোনটা	কেন ভুল
A	🥈 Silver	"Cleaned data" মানে Silver। এখান থেকে aggregate করে Gold বানানো হয় (Silver → Gold)।
B	🥇 Gold	Aggregate করা data হলো Gold। Dashboard সাধারণত Gold table থেকে চলে।
C	Raw source (stream/file)	এই job টা Bronze table বানায় , Bronze থেকে পড়ে না (Source → Bronze)।
D	🥈 Silver / 🥇 Gold	ML-এর feature set সাধারণত পরিষ্কার বা aggregate করা data থেকে বানানো হয়, raw Bronze থেকে নয়।

🔑 মনে রাখার উপায়

Job	Source → Target
Raw data ingest	Source → 🥉 Bronze
Clean, parse, enrich, filter, deduplicate	🥉 Bronze → 🥈 Silver ✅
Aggregate, summary statistics	🥈 Silver → 🥇 Gold
Dashboard, report, ML	🥇 Gold (বা Silver) থেকে

💡 **Bronze = raw, Silver = clean, Gold = aggregated** 💡 **Bronze-কে source হিসেবে ব্যবহার করে এমন job = Silver বানানোর job** (cleaning/enrichment)

সংক্ষেপে: timestamp parse করে data enrich করা হলো **Bronze** → **Silver** ধাপের কাজ, তাই এই job Bronze table-কে source হিসেবে ব্যবহার করে। উত্তর **E**।

Question 24

QUESTION

Which of the following data workloads will utilize a Silver table as its source?

- A. A job that enriches data by parsing its timestamps into a human-readable format
- B. A job that queries aggregated data that already feeds into a dashboard
- C. A job that ingests raw data from a streaming source into the Lakehouse
- D. A job that aggregates cleaned data to create standard summary statistics
- E. A job that cleans data by removing malformed records

✓ সঠিক উত্তর: D

ব্যাখ্যা

Medallion Architecture আবার একবার দেখে নেওয়া যাক:

Raw Source → 🥉 Bronze → 🥈 Silver → 🥇 Gold → Dashboard / ML
(file, stream) (raw data) (clean, enriched) (aggregated)

প্রশ্নে জানতে চাওয়া হয়েছে কোন job **Silver থেকে data পড়ে**। Silver থেকে data পড়ে **Gold বানানো** হয়, আর Gold-এর কাজ হলো data **aggregate করা** (summary, statistics, business-level report)।

Option **D**-তে দুটো জিনিস Silver → Gold-কে চিনিয়ে দেয়:

- **"Cleaned data"** → পরিষ্কার data থাকে **Silver**-এ (source)
- **"Aggregates ... summary statistics"** → aggregate করা data যায় **Gold**-এ (target)

```
from pyspark.sql.functions import avg, count, sum

(spark.table("silver_sales")                                     # 🥈 Silver থেকে পড়া
  .groupBy("region")
  .agg(count("*").alias("total_orders"),
        sum("amount").alias("total_sales"),
        avg("amount").alias("avg_sales"))                       # summary statistics
  .write
  .mode("overwrite")
  .saveAsTable("gold_sales_summary"))                           # 🥇 Gold-এ লেখা
```

❌ বাকি option গুলো কেন ভুল

Option	Source কোনটা	কেন ভুল
A	🥉 Bronze	Timestamp parse করে enrich করা হলো Bronze → Silver -এর কাজ। এটা Silver বানায়, Silver থেকে পড়ে না। (আগের প্রশ্নের উত্তর এটাই ছিল।)
B	🥇 Gold	Dashboard-এ ব্যবহার হওয়া aggregate data হলো Gold । তাই এই job Gold থেকে পড়ে।
C	Raw source	Raw data ingest করা হলো Source → Bronze । এটা Bronze বানায়।
E	🥉 Bronze	Malformatted record বাদ দিয়ে data পরিষ্কার করাও Bronze → Silver -এর কাজ।

🔑 মনে রাখার উপায়

Job	Source → Target
Raw data ingest	Source → 🥉 Bronze
Clean, parse, enrich, filter, malformed record বাদ	🥉 Bronze → 🥈 Silver
Aggregate, summary statistics	🥈 Silver → 🥇 Gold ✅
Dashboard query	🥇 Gold থেকে

Keyword দেখে চেনার উপায়:

Keyword	Layer
"raw", "ingest"	Source → Bronze
"clean", "parse", "enrich", "filter", "deduplicate", "malformatted"	Bronze → Silver
"aggregate", "summary", "statistics", "business-level"	Silver → Gold
"dashboard", "report", "insights"	Gold থেকে

সংক্ষেপে: পরিষ্কার data aggregate করে summary statistics বানানো হলো **Silver** → **Gold** ধাপের কাজ, তাই এই job Silver table-কে source হিসেবে ব্যবহার করে। উত্তর **D**।

Question 25

QUESTION

Which of the following Structured Streaming queries is performing a hop from a Bronze table to a Silver table?

A.

```
(spark.table("sales")
  .groupBy("store")
  .agg(sum("sales"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("aggregatedSales")
)
```

B.

```
(spark.table("sales")
  .agg(sum("sales"),
       sum("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("aggregatedSales")
)
```

C.

```
(spark.table("sales")
  .withColumn("avgPrice", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("cleanedSales")
)
```

D.

```
(spark.readStream.load(rawSalesLocation)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("uncleanedSales")
)
```

E.

```
(spark.read.load(rawSalesLocation)
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("append")
  .table("uncleanedSales")
)
```

✓ সঠিক উত্তর: C

ব্যাখ্যা

Bronze → **Silver** hop মানে raw data-কে পরিষ্কার করা বা enrich করা। এখানে **row-এর সংখ্যা মোটামুটি একই থাকে**, শুধু data আরও কাজের হয়ে ওঠে।

Option **C**-তে তিনটা জিনিস Bronze → Silver hop-কে চিনিয়ে দেয়:

চিহ্ন	অর্থ
<code>.withColumn("avgPrice", ...)</code>	প্রতিটা row-এ একটা নতুন হিসাব করা column যোগ হচ্ছে, অর্থাৎ data enrich হচ্ছে (aggregation নয়)
<code>.outputMode("append")</code>	শুধু নতুন row যোগ হয়। Row-ধরে কাজ করা transformation-এর ক্ষেত্রে এটাই স্বাভাবিক
<code>.table("cleanedSales")</code>	Target table-এর নামে "cleaned" , অর্থাৎ Silver

তাই C হলো `sales` (Bronze) → `cleanedSales` (Silver)।

✗ বাকি option গুলো কেন ভুল

Option	Hop	কেন ভুল
A	 Silver →  Gold	<code>groupBy("store").agg(sum(...))</code> দিয়ে store অনুযায়ী aggregate করা হচ্ছে। <code>outputMode("complete")</code> আর target-এর নাম <code>aggregatedSales</code> , এগুলো সবই Gold -এর চিহ্ন।
B	 Silver →  Gold	পুরো table-এর উপর <code>agg(sum(...))</code> , অর্থাৎ aggregation । এটাও Gold বানানোর কাজ।
D	Raw →  Bronze	Raw location থেকে <code>readStream.load()</code> দিয়ে data পড়ে কোনো পরিবর্তন ছাড়াই লেখা হচ্ছে। Target-এর নাম <code>uncleanedSales</code> , মানে এটা Bronze বানাচ্ছে (ingestion)।
E	✗ Error	এটাও Raw → Bronze-এর চেষ্টা, কিন্তু <code>spark.read (batch)</code> দিয়ে পড়ে <code>.writeStream</code> চালানো যায় না। Batch DataFrame-এ writeStream চলে না , তাই error দেবে।

🔑 মনে রাখার উপায়

Code দেখে hop চেনার উপায়:

চিহ্ন	Hop
Raw path থেকে <code>readStream.load()</code> / <code>cloudFiles</code> , কোনো transformation নেই	Raw → 🥉 Bronze
<code>withColumn</code> , <code>filter</code> , <code>cast</code> , <code>parse</code> , <code>join</code> , <code>deduplicate</code> + <code>append</code> mode	🥉 Bronze → 🥈 Silver ✓
<code>groupBy</code> , <code>agg</code> , <code>sum</code> , <code>count</code> + <code>complete</code> mode	🥈 Silver → 🥇 Gold

Output mode:

Mode	কখন ব্যবহার হয়
<code>append</code>	শুধু নতুন row যোগ হয় (Bronze, Silver)
<code>complete</code>	প্রতিবার পুরো ফলাফল নতুন করে লেখা হয় (aggregation, অর্থাৎ Gold)
<code>update</code>	শুধু বদলে যাওয়া row গুলো আপডেট হয়

💡 **Row-ধরে পরিবর্তন (withColumn/filter) = Silver, Aggregation (groupBy/agg) = Gold**

ছোট একটা বিষয়: streaming read-এর জন্য সঠিক syntax হলো `spark.readStream.table("sales")` । প্রশ্নে `spark.table()` লেখা আছে, তবে পরীক্ষার জন্য মূল বিষয় হলো transformation-এর ধরন।
সংক্ষেপে: `withColumn` দিয়ে data enrich করে `append` mode-এ `cleanedSales` -এ লেখা হলো **Bronze** → **Silver** hop। তাই উত্তর **C**।

Question 26

QUESTION

Which of the following benefits does **Lakeflow Spark Declarative Pipelines** provide for ELT pipelines over standard data pipelines that utilize Spark and Delta Lake on Databricks?

- A. The ability to declare and maintain data table dependencies
- B. The ability to write pipelines in Python and/or SQL
- C. The ability to access previous versions of data tables
- D. The ability to automatically scale compute resources
- E. The ability to perform batch and streaming queries

✓ সঠিক উত্তর: A

ব্যাখ্যা

Delta Live Tables (DLT) হলো একটা **declarative** framework। এখানে আপনি শুধু বলে দেন **কোন table কোন table থেকে, কীভাবে তৈরি হবে**। কোন table আগে চলবে, কোনটা পরে, সেই ক্রম DLT নিজেই ঠিক করে নেয়।

DLT কীভাবে dependency সামলায়?

- প্রতিটা table-এর সংজ্ঞায় আপনি অন্য table-কে reference করেন (যেমন `LIVE.bronze_sales` বা `dlt.read("bronze_sales")`)
- DLT এই reference গুলো দেখে নিজে থেকেই একটা **dependency graph (DAG)** বানায়
- সঠিক ক্রমে table গুলো তৈরি ও update করে
- UI-তে pipeline-এর পুরো graph চোখে দেখা যায়

```
-- Bronze
CREATE OR REFRESH STREAMING TABLE bronze_sales
AS SELECT * FROM cloud_files("/raw/sales", "json");

-- Silver (Bronze-এর উপর নির্ভরশীল)
CREATE OR REFRESH STREAMING TABLE silver_sales
AS SELECT *, sales / units AS avg_price
FROM STREAM(LIVE.bronze_sales);

-- Gold (Silver-এর উপর নির্ভরশীল)
CREATE OR REFRESH MATERIALIZED VIEW gold_sales
AS SELECT store, SUM(sales) AS total_sales
FROM LIVE.silver_sales
GROUP BY store;
```

bronze_sales → silver_sales → gold_sales
(DLT নিজে থেকে এই ক্রম বুঝে নেয়)

সাধারণ Spark pipeline-এ এই ক্রম আপনাকে **নিজে হাতে** সামলাতে হয়: আলাদা আলাদা notebook/job বানানো, কোনটা আগে চলবে ঠিক করা, fail করলে কী হবে সেটা দেখা ইত্যাদি।

❌ বাকি option গুলো কেন ভুল

এগুলো সবই সাধারণ Spark + Delta Lake pipeline-এও **আগে থেকেই পাওয়া যায়**, তাই DLT-এর **বাড়তি** সুবিধা নয়:

Option	কেন ভুল
B	সাধারণ Databricks notebook-এও Python আর SQL দুটোতেই pipeline লেখা যায়।
C	আগের version দেখা হলো Delta Lake-এর Time Travel সুবিধা। DLT ছাড়াই এটা পাওয়া যায়।
D	সাধারণ Databricks cluster-এও autoscaling চালু করা যায়। (DLT-তে enhanced autoscaling আছে, কিন্তু autoscaling নিজে DLT-এর একার সুবিধা নয়।)
E	Spark Structured Streaming দিয়ে সাধারণ pipeline-এও batch আর streaming দুটোই চালানো যায়।

🔑 মনে রাখার উপায়

DLT-এর নিজস্ব (বাড়তি) সুবিধা:

সুবিধা	বর্ণনা
Declarative pipeline	"কী বানাতে হবে" বলে দিলেই হয়, "কীভাবে" নিয়ে ভাবতে হয় না
Table dependency সামলানো ✅	নিজে থেকে DAG বানায় আর ক্রম ঠিক করে
Data quality (Expectations)	<code>CONSTRAINT ... EXPECT ...</code> দিয়ে data যাচাই
Automatic error handling ও retry	Fail করলে নিজে থেকে আবার চেষ্টা করে
Pipeline monitoring ও lineage	UI-তে graph আর event log

যেগুলো DLT-এর একার সুবিধা নয়:

সুবিধা	আসলে কোথা থেকে আসে
Time Travel	Delta Lake
Batch + Streaming	Spark Structured Streaming
Python / SQL	Databricks notebook
Autoscaling	Databricks cluster

সংক্ষেপে: DLT table-এর মধ্যে **dependency নিজে থেকে ঘোষণা ও সামলানোর** সুবিধা দেয়, যা সাধারণ Spark pipeline-এ নিজে হাতে করতে হয়। তাই উত্তর **A**।

Question 27

QUESTION

A data engineer has three notebooks in an ELT pipeline. The notebooks need to be executed in a specific order for the pipeline to complete successfully. The data engineer would like to use **Lakeflow Spark Declarative Pipelines** to manage this process.

Which of the following steps must the data engineer take as part of implementing this pipeline using **Lakeflow Spark Declarative Pipelines** ?

- A. They need to create a **Lakeflow Spark Declarative Pipelines** pipeline from the Data page.
- B. They need to create a **Lakeflow Spark Declarative Pipelines** pipeline from the **Lakeflow Jobs page**.
- C. They need to create a **Lakeflow Spark Declarative Pipelines** pipeline from the Compute page.
- D. They need to refactor their notebook to use Python and the dlt library.
- E. They need to refactor their notebook to use SQL and **CREATE OR REFRESH MATERIALIZED VIEW** keyword.

✓ সঠিক উত্তর: B

ব্যাখ্যা

Databricks-এ **Delta Live Tables pipeline** বানানো হয় **Jobs (Workflows) page** থেকে:

Sidebar → Workflows (Jobs) → Delta Live Tables tab → Create Pipeline

Pipeline বানানোর সময় যা যা সেট করতে হয়:

- Pipeline-এর **নাম**
- কোন **notebook** গুলো (source code) pipeline-এ থাকবে (একাধিক notebook দেওয়া যায়)
- Table গুলো কোথায় save হবে (**target schema / storage location**)
- **Pipeline mode**: Triggered (একবার চালিয়ে বন্ধ) নাকি Continuous (সারাক্ষণ চলে)
- **Cluster** setting

এরপর DLT তিনটা notebook-এর table reference দেখে **নিজে থেকেই ঠিক করে নেয় কোনটা আগে চলবে** (dependency graph)। তাই আলাদা করে ক্রম ঠিক করে দিতে হয় না।

💡 নতুন Databricks UI-তে এই অংশের নাম **"Workflows"** বা **"Jobs & Pipelines"**। কিন্তু পরীক্ষার প্রশ্নে এটাকে **Jobs page** বলা হয়।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. Data page	Data page (Data Explorer / Catalog Explorer) দিয়ে database, table আর permission দেখা ও সামলানো হয়। এখান থেকে pipeline বানানো যায় না।
C. Compute page	Compute page-এ cluster আর SQL warehouse বানানো ও সামলানো হয়। DLT নিজের cluster নিজে সামলায়, pipeline এখান থেকে বানানো হয় না।
D. Python + dlt	DLT Python বা SQL, যেকোনো একটা দিয়েই লেখা যায়। Python ব্যবহার করা বাধ্যতামূলক নয়, তাই এটা "must" ধাপ নয়।
E. SQL + CREATE LIVE TABLE	একই কারণে ভুল। SQL ব্যবহার করা বাধ্যতামূলক নয়, Python দিয়েও লেখা যায়।

⚠️ D আর E-তে একটা কথা ঠিক: সাধারণ notebook-কে DLT-তে চালাতে হলে **DLT syntax-এ বদলাতে হয়**। কিন্তু কোন ভাষায় লিখবেন, সেটা আপনার ইচ্ছা। তাই নির্দিষ্ট একটা ভাষা ব্যবহারের কথা বলা option গুলো "must" হিসেবে ভুল।

🔑 মনে রাখার উপায়

Databricks Page	কাজ
Jobs / Workflows	Job আর DLT pipeline বানানো ও schedule করা ✅
Data / Catalog	Database, table, permission দেখা
Compute	Cluster ও SQL warehouse সামলানো
SQL Editor	SQL query চালানো

DLT syntax (যেকোনো একটা ভাষায়):

SQL	Python
CREATE OR REFRESH LIVE TABLE t AS SELECT ...	@dlt.table decorator + function
CREATE OR REFRESH STREAMING TABLE t ...	@dlt.table + spark.readStream
LIVE.table_name দিয়ে reference	dlt.read("table_name") দিয়ে reference

সংক্ষেপে: Delta Live Tables pipeline বানানো হয় **Jobs (Workflows) page** থেকে। তাই উত্তর **B**।

Question 28

QUESTION

A data engineer has written the following query:

```
SELECT *  
FROM json.`/path/to/json/file.json`;
```

The data engineer asks a colleague for help to convert this query for use in a **Lakeflow Spark Declarative Pipelines (SDP)** pipeline. The query should create the first table in the **SDP** pipeline.

Which of the following describes the change the colleague needs to make to the query?

- A. They need to add a COMMENT line at the beginning of the query.
- B. They need to add a **CREATE OR REFRESH MATERIALIZED VIEW** table_name AS line at the beginning of the query.
- C. They need to add a live. prefix prior to json. in the FROM line.
- D. They need to add a CREATE DELTA LIVE TABLE table_name AS line at the beginning of the query.
- E. They need to add the cloud_files(...) wrapper to the JSON file path

✓ সঠিক উত্তর: B

```
CREATE LIVE TABLE table_name AS  
SELECT *  
FROM json.`/path/to/json/file.json`;
```

ব্যাখ্যা

DLT-তে প্রতিটি table **declarative** ভাবে সংজ্ঞায়িত করতে হয়। অর্থাৎ বলে দিতে হয় **কোন নামে table টা তৈরি হবে, আর তার data কোন query থেকে আসবে।**

শুধু একটি **SELECT** query DLT-তে কোনো table বানায় না। DLT-কে জানাতে হয় যে এই query-র ফলাফল দিয়ে একটি table বানাতে হবে। সেজন্য query-র শুরুতে লাগে:

```
CREATE LIVE TABLE table_name AS
```

এখানে:

- **CREATE LIVE TABLE** → DLT-কে বলে দেয় এটা pipeline-এর একটি table
- **table_name** → table-এর নাম
- **AS SELECT ...** → table-এর data কোথা থেকে আসবে

💡 নতুন DLT syntax-এ **CREATE OR REFRESH LIVE TABLE** বা **CREATE OR REFRESH MATERIALIZED VIEW** লেখা হয়। কিন্তু পরীক্ষার জন্য মূল কথা হলো: **DLT table বানাতে CREATE LIVE TABLE ... AS |**

❌ **বাকি option গুলো কেন ভুল**

Option	কেন ভুল
A	<code>COMMENT</code> শুধু table-এর বর্ণনা যোগ করে (ঐচ্ছিক)। এটা যোগ করলেও table তৈরি হয় না।
C	<code>live.</code> prefix ব্যবহার হয় pipeline-এর ভেতরে আগে থেকে থাকা আরেকটা DLT table-কে reference করতে (যেমন <code>LIVE.bronze_table</code>)। এটা প্রথম table , তাই reference করার মতো কোনো DLT table এখনো নেই। আর <code>live.json.</code> লেখা ভুল syntax।
D	SQL-এ <code>CREATE DELTA LIVE TABLE</code> বলে কোনো syntax নেই । সঠিক syntax হলো <code>CREATE LIVE TABLE</code> ।
E	<code>cloud_files()</code> হলো DLT SQL-এ Auto Loader ব্যবহারের উপায়। এটা incremental ingestion-এর জন্য, আর তখন <code>STREAMING LIVE TABLE</code> লাগে। কিন্তু শুধু এটা যোগ করলেও <code>CREATE ... TABLE</code> line ছাড়া table তৈরি হবে না। তাই এটা বাস্তবতামূলক পরিবর্তন নয় ।

🔑 **মনে রাখার উপায়**

DLT SQL Syntax	কাজ
<code>CREATE LIVE TABLE t AS SELECT ...</code>	সাধারণ DLT table (batch / materialized view) ✓
<code>CREATE STREAMING LIVE TABLE t AS SELECT ...</code>	Streaming DLT table (incremental)
<code>FROM cloud_files("/path", "json")</code>	Auto Loader দিয়ে file পড়া (streaming table-এ)
<code>FROM LIVE.other_table</code>	Pipeline-এর অন্য DLT table reference করা
<code>FROM STREAM(LIVE.other_table)</code>	অন্য DLT table থেকে streaming read

সম্পূর্ণ উদাহরণ:

```
-- প্রথম table (raw file থেকে)
CREATE LIVE TABLE bronze_data AS
SELECT * FROM json.`/path/to/json/file.json`;

-- দ্বিতীয় table (প্রথম table থেকে, LIVE. prefix দিয়ে)
CREATE LIVE TABLE silver_data AS
SELECT * FROM LIVE.bronze_data
WHERE id IS NOT NULL;
```

সংক্ষেপে: DLT-তে table বানাতে query-র শুরুতে `CREATE LIVE TABLE table_name AS` যোগ করতে হয়। তাই উত্তর **B**।

Question 29

QUESTION

A dataset has been defined using **Lakeflow Spark Declarative Pipelines** and includes an expectations clause:

```
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01')
```

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.
- B. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.
- C. Records that violate the expectation cause the job to fail.
- D. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.
- E. Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.

✓ সঠিক উত্তর: A

ব্যাখ্যা

DLT-তে **Expectations** দিয়ে data quality যাচাই করা হয়। শর্ত না মানা record-এর সাথে কী করা হবে, তা ঠিক হয় **ON VIOLATION** অংশ দিয়ে।

প্রশ্নের constraint-এ কোনো **ON VIOLATION** অংশ নেই:

```
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01')
--
                                     † ON VIOLATION নেই
```

ON VIOLATION না থাকলে DLT **default আচরণ** দেখায়, যার নাম **"warn"**:

1. শর্ত না মানা record-ও **target table-এ যোগ হয়** (বাদ পড়ে না)
2. কয়টা record শর্ত মানেনি, সেই হিসাব **event log-এ রেকর্ড হয়** (data quality metrics)
3. Pipeline **চলতে থাকে**, fail করে না

এতে data হারায় না, আবার data quality-র উপর নজরও রাখা যায়।

🔑 তিন ধরনের Expectation আচরণ

Syntax	শর্ত না মানলে কী হয়	Option
EXPECT (...)	Record রাখা হয়, event log-এ রেকর্ড (warn, default)	✅ A
EXPECT (...) ON VIOLATION DROP ROW	Record বাদ পড়ে, event log-এ রেকর্ড	B
EXPECT (...) ON VIOLATION FAIL UPDATE	Pipeline fail করে	C

```
-- 1. Warn (default): record থাকে
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01')

-- 2. Drop: record বাদ পড়ে
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW

-- 3. Fail: pipeline থেমে যায়
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION FAIL UPDATE
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	Record বাদ পড়ে শুধু ON VIOLATION DROP ROW থাকলে। এখানে সেটা নেই।
C	Job fail করে শুধু ON VIOLATION FAIL UPDATE থাকলে। এখানে সেটা নেই।
D	DLT নিজে থেকে target table-এ কোনো নতুন "invalid" field যোগ করে না। Validation-এর ফলাফল থাকে event log-এ, table-এ নয়।
E	DLT-তে নিজে থেকে কোনো quarantine table হয় না। চাইলে আলাদা একটা table বানিয়ে উল্টো শর্ত দিয়ে (যেমন NOT (timestamp > '2020-01-01')) এমন কিছু করা যায়, কিন্তু সেটা default আচরণ নয়।

🔑 মনে রাখার উপায়

ON VIOLATION	ফলাফল	মনে রাখার শব্দ
(কিছু না)	রাখা + log করা	Warn ⚠️
DROP ROW	বাদ দাও + log করা	Drop 🗑️
FAIL UPDATE	থামিয়ে দাও	Fail ❌

💡 ON VIOLATION নেই = record থাকে, শুধু event log-এ রেকর্ড হয়।

সংক্ষেপে: ON VIOLATION ছাড়া expectation হলো default warn mode। এতে শর্ত না মানা record table-এ যোগ হয় আর event log-এ invalid হিসেবে রেকর্ড হয়। তাই উত্তর A।

Question 30

QUESTION

A **Lakeflow Spark Declarative Pipeline** pipeline includes two datasets defined using **STREAMING TABLE**. Three datasets are defined against Delta Lake table sources using **MATERIALIZED VIEW**.

The table is configured to run in Development mode using the Triggered Pipeline Mode.

Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

- A. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.
- B. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.
- C. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist after the pipeline is stopped to allow for additional testing.
- D. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.
- E. All datasets will be updated continuously and the pipeline will not shut down. The compute resources will persist with the pipeline.

✓ সঠিক উত্তর: D

ব্যাখ্যা

এই প্রশ্নের উত্তর দুটো setting-এর উপর নির্ভর করে: **Pipeline mode** আর **Environment mode**। দুটোকে আলাদা করে দেখতে হবে।

১. Pipeline Mode: Triggered → কতবার চলবে

Mode	আচরণ
Triggered ✓	সব dataset একবার update হয়, তারপর pipeline বন্ধ হয়ে যায়
Continuous	Pipeline সারাক্ষণ চলে, নতুন data এলেই update করে

→ Triggered, তাই "সব dataset একবার update হবে আর pipeline বন্ধ হবে"। এতে উত্তর A বা D-তে নেমে আসে।

২. Environment Mode: Development → Cluster-এর কী হবে

Mode	Cluster	Retry
Development ✓	Update শেষ হলেও cluster চালু থাকে (testing-এর জন্য বারবার নতুন cluster চালু করতে হয় না)	Error হলে retry করে না , যাতে তাড়াতাড়ি ভুল ধরা যায়
Production	Update শেষ হলে cluster সাথে সাথে বন্ধ হয়	Error হলে নিজে থেকে retry করে (এমনকি cluster restart করে)

→ Development mode, তাই **"compute resource চালু থাকবে"**। এতে উত্তর হয় **D**।

💡 Development mode-এ cluster সাধারণত ডিফল্টভাবে **প্রায় ২ ঘণ্টা** চালু থাকে (pipelines.clusterShutdown.delay setting দিয়ে বদলানো যায়)।

💡 **STREAMING LIVE TABLE** থাকলেও Triggered mode-এ সেগুলো **শুধু available নতুন data একবার process করে থেমে যায়**। Streaming table মানেই pipeline সারাক্ষণ চলবে, এমন নয়।

✗ বাকি option গুলো কেন ভুল

Option	Pipeline অংশ	Compute অংশ	কেন ভুল
A	✓ একবার update	✗ Terminate	Compute terminate হয় Production mode -এ। এখানে Development mode।
B	✗ নির্দিষ্ট সময় পর পর	✗ Stop করলে terminate	বারবার update হয় Continuous mode -এ। এখানে Triggered।
C	✗ নির্দিষ্ট সময় পর পর	✓ চালু থাকে	Compute অংশ ঠিক, কিন্তু pipeline অংশ Continuous mode -এর আচরণ।
E	✗ সারাক্ষণ চলে	—	এটা Continuous mode -এর আচরণ।

🔑 মনে রাখার উপায়

	Development (cluster চালু থাকে)	Production (cluster বন্ধ হয়)
Triggered (একবার চলে)	একবার update → pipeline বন্ধ → cluster চালু ✓ (D)	একবার update → pipeline বন্ধ → cluster বন্ধ (A)
Continuous (সারাক্ষণ চলে)	সারাক্ষণ update → cluster চালু	সারাক্ষণ update → cluster চালু

💡 **Triggered = একবার, Continuous = সারাক্ষণ** 💡 **Development = cluster রেখে দাও, retry নয়, Production = cluster বন্ধ করো, retry করো**

সংক্ষেপে: **Triggered** mode-এ সব dataset একবার update হয়ে pipeline থামে, আর **Development** mode-এ testing-এর জন্য cluster চালু থাকে। তাই উত্তর **D**।

Question 31

QUESTION

A data engineer has a Job with multiple tasks that runs nightly. One of the tasks unexpectedly fails during 10 percent of the runs.

Which of the following actions can the data engineer perform to ensure the Job completes each night while minimizing compute costs?

- A. They can institute a retry policy for the entire Job
- B. They can observe the task as it runs to try and determine why it is failing
- C. They can set up the Job to run multiple times ensuring that at least one will complete
- D. They can institute a retry policy for the task that periodically fails
- E. They can utilize a Jobs cluster for each of the tasks in the Job

✓ সঠিক উত্তর: D

ব্যাখ্যা

প্রশ্নে দুটো শর্ত আছে:

1. Job প্রতি রাতে সফলভাবে শেষ হতে হবে
2. Compute খরচ কম রাখতে হবে

Task-level retry policy দুটো শর্তই পূরণ করে:

- Fail করা task-টা নিজে থেকে আবার চলে → Job সফলভাবে শেষ হয় ✓
- শুধু fail করা task-টাই আবার চলে, আগে সফল হওয়া task গুলো নয় → compute খরচ সবচেয়ে কম ✓

Task 1 ✓ → Task 2 ✓ → Task 3 ✗ → (retry) Task 3 ✓ → Task 4 ✓

↑
শুধু এই task টাই আবার চলে

কীভাবে সেট করতে হয়: Workflows → Job → নির্দিষ্ট Task → **Retries** → retry-র সংখ্যা ও বিরতি (interval) ঠিক করে দেওয়া

যেহেতু task টা মাত্র ১০% সময় fail করে, তাই একবার বা দুইবার retry করলেই প্রায় সবসময় সফল হবে।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	পুরো Job আবার চালালে আগে সফল হওয়া task গুলোও আবার চলবে। এতে বাড়তি compute খরচ হয়। (তাহাড়া Databricks-এ retry policy আসলে task-level-এ সেট করা হয়।)
B	কারণ খুঁজে বের করা ভালো অভ্যাস, কিন্তু এতে প্রতি রাতে Job শেষ হওয়া নিশ্চিত হয় না। আর বসে বসে নজর রাখা কোনো automated সমাধানও নয়।
C	Job কয়েকবার চালালে কয়েক গুণ compute খরচ হবে। এটা সবচেয়ে বেশি খরচের পথ।
E	Jobs cluster (all-purpose cluster-এর চেয়ে সস্তা) খরচ কমাতে সাহায্য করে, কিন্তু task fail হওয়ার সমস্যা মেটায় না। Job শেষ হওয়া নিশ্চিত হয় না।

🔑 মনে রাখার উপায়

পদ্ধতি	Job সফল হওয়া নিশ্চিত?	Compute খরচ
Task-level retry ✅	✅ হ্যাঁ	🟢 সবচেয়ে কম (শুধু fail করা task)
পুরো Job retry	✅ হ্যাঁ	🟡 বেশি (সব task আবার চলে)
Job একাধিকবার চালানো	✅ হ্যাঁ	🔴 অনেক বেশি
নজর রাখা	❌ না	—
Jobs cluster	❌ না	🟢 কম, কিন্তু সমস্যার সমাধান নয়

💡 সমস্যা যেখানে, সমাধানও সেখানে: শুধু যে task fail করে, retry দিতে হবে সেটাকেই।

সংক্ষেপে: শুধু মাঝে মাঝে fail করা task-টার জন্য retry policy দিলে Job সফল হয় আর খরচও সবচেয়ে কম থাকে। তাই উত্তর D।

Question 32

QUESTION

A data engineer has set up two Jobs that each run nightly. The first Job starts at 12:00 AM, and it usually completes in about 20 minutes. The second Job depends on the first Job, and it starts at 12:30 AM. Sometimes, the second Job fails when the first Job does not complete by 12:30 AM.

Which of the following approaches can the data engineer use to avoid this problem?

- A. They can utilize multiple tasks in a single job with a linear dependency
- B. They can use cluster pools to help the Jobs run more efficiently
- C. They can set up a retry policy on the first Job to help it run more quickly
- D. They can limit the size of the output in the second Job so that it will not fail as easily
- E. They can set up the data to stream from the first Job to the second Job

✓ પ્રાથમિક ઉત્તર: A

व्याख्या

সমস্যার মূল কারণ কী? দুটো Job নির্দিষ্ট সময় ধরে আলাদাভাবে schedule করা। দ্বিতীয় Job জানে না প্রথম Job শেষ হয়েছে কিনা, সে শুধু ঘড়ি দেখে ১২:৩০-এ শুরু হয়ে যায়। প্রথম Job কোনো দিন ৩০ মিনিটের বেশি নিলে দ্বিতীয় Job অসম্পূর্ণ data নিয়ে চলে আর fail করে।

সমাধান: একটা Job, দুটো task, dependency সহ দুটো কাজকে একটা Job-এর ভেতরে দুটো task হিসেবে রাখতে হবে, আর Task 2-কে Task 1-এর উপর নির্ভরশীল (depends on) করে দিতে হবে:

❌ আগে (সময় ধরে আলাদা schedule):

Job 1: 12:00 AM শুরু → (কখনো ২০ মিনিট, কখনো ৪০ মিনিট)

Job 2: 12:30 AM শুরু → Job 1 শেষ না হলে fail ❌

✓ পরে (একটা Job, linear dependency):

Job (12:00 AM): Task 1 $\rightarrow$ Task 2

↑

Task 1 শেষ হলেই তবে শুরু হবে

এতে:

- Task 1 যতক্ষণই নিক, **Task 2 শুধু Task 1 সফলভাবে শেষ হওয়ার পরেই শুরু হবে**
- Task 1 fail করলে Task 2 **চলাবেই না** (অসম্পূর্ণ data নিয়ে কাজ হবে না)
- Timing নিয়ে আর কোনো অনুমান করতে হয় না

কীভাবে সেট করতে হয়: Workflows → Job → **Add task** → নতুন task-এর **"Depends on"** অংশে প্রথম task টা বেছে দেওয়া

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	Cluster pool আগে থেকে চালু রাখা machine দিয়ে cluster তড়াতাড়ি চালু করতে সাহায্য করে। এতে Job হয়তো একটু দ্রুত শুরু হবে, কিন্তু প্রথম Job ১২:৩০-এর মধ্যে শেষ হবে, এমন নিশ্চয়তা নেই । মূল সমস্যা (dependency না থাকা) থেকেই যায়।
C	Retry policy fail করা task আবার চালায় , কিন্তু Job-কে দ্রুত করে না । বরং retry হলে প্রথম Job আরও বেশি সময় নেবে, ফলে সমস্যা আরও বাড়বে।
D	দ্বিতীয় Job fail করে কারণ প্রথম Job তখনো শেষ হয়নি , output-এর size-এর জন্য নয়। এটা কারণের সাথে সম্পর্কহীন সমাধান।
E	Streaming-এ বদলানো অপ্রয়োজনীয়ভাবে জটিল আর খরচও বেশি (cluster সারাক্ষণ চালু রাখতে হতে পারে)। সমস্যাটা শুধু ক্রম নিয়ে, আর সেটা task dependency দিয়েই সহজে মেটে।

🔑 মনে রাখার উপায়

সমস্যা	সমাধান
এক কাজ আরেক কাজের উপর নির্ভরশীল	একটা Job-এ multiple task + dependency ✅
একটা task মাঝে মাঝে fail করে	Task-level retry policy
Cluster চালু হতে দেরি হয়	Cluster pool
খরচ কমানো	Jobs cluster (all-purpose-এর বদলে)

💡 **সময় ধরে schedule ≠ dependency**। একটা কাজ আরেকটার উপর নির্ভর করলে **task dependency** ব্যবহার করতে হবে, ঘড়ির সময় নয়।

সংক্ষেপে: দুটো কাজকে **একটা Job-এর ভেতরে task হিসেবে রেখে linear dependency** দিলে দ্বিতীয় task শুধু প্রথমটা শেষ হলেই শুরু হবে। তাই উত্তর **A**।

Question 33

QUESTION

A data engineer has set up a notebook to automatically process using a Job. The data engineer's manager wants to version control the schedule due to its complexity.

Which of the following approaches can the data engineer use to obtain a version-controllable configuration of the Job's schedule?

- A. They can link the Job to notebooks that are a part of a **Databricks Git Folder**.
- B. They can submit the Job once on a Job cluster.
- C. They can download the JSON description of the Job from the Job's page.
- D. They can submit the Job once on an all-purpose cluster.
- E. They can download the XML description of the Job from the Job's page.

✓ সঠিক উত্তর: C

ব্যাখ্যা

Databricks-এ প্রতিটা Job-এর পুরো configuration (schedule, task, cluster, parameter, retry policy, notification ইত্যাদি) একটা **JSON** format-এ রাখা থাকে।

কীভাবে পাওয়া যায়: Workflows → Job → উপরে ডান দিকের : (kebab menu) → **View JSON** (বা "**View YAML/JSON**") → copy / download

JSON-এ schedule দেখতে কেমন:

```
{
  "name": "nightly_processing_job",
  "schedule": {
    "quartz_cron_expression": "0 0 2 ? * MON-FRI",
    "timezone_id": "Asia/Dhaka",
    "pause_status": "UNPAUSED"
  },
  "tasks": [
    {
      "task_key": "process_data",
      "notebook_task": {
        "notebook_path": "/Repos/team/project/process_notebook"
      }
    }
  ]
}
```

এই JSON file টা **Git-এ commit** করে রাখলে:

- Schedule-এর প্রতিটা পরিবর্তনের **ইতিহাস** থাকে
- দরকার হলে **আগের version-এ ফিরে যাওয়া** যায়
- এই JSON দিয়ে Jobs API, Databricks CLI বা Terraform/Databricks Asset Bundles ব্যবহার করে **একই Job আবার বানানো** যায়

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Repo-তে রাখলে notebook-এর code version control-এ থাকে, কিন্তু Job-এর schedule থাকে না । Schedule হলো Job-এর configuration, notebook-এর অংশ নয়।
B	Job cluster-এ submit করলে Job শুধু একবার চলে । এতে কোনো version control করার মতো configuration পাওয়া যায় না।
D	একই কারণে ভুল। কোন cluster-এ চলছে তাতে schedule-এর version control-এর কিছু যায় আসে না।
E	Databricks Job-এর configuration XML format-এ পাওয়া যায় না । Databricks JSON ব্যবহার করে।

🔑 মনে রাখার উপায়

কী version control করতে চান	উপায়
Notebook-এর code	Databricks Repos (Git integration)
Job-এর configuration / schedule	Job page থেকে JSON download ✅
পুরো project (code + job + pipeline)	Databricks Asset Bundles (YAML) / Terraform

💡 **Code → Repos, Job config → JSON** 💡 Databricks Job = **JSON** (XML নয়)

সংক্ষেপে: Job-এর schedule সহ পুরো configuration **JSON** হিসেবে download করে Git-এ রাখা যায়। তাই উত্তর **C**।

Question 34

QUESTION

A data analyst has noticed that their Databricks SQL queries are running too slowly. They claim that this issue is affecting all of their sequentially run queries. They ask the data engineering team for help. The data engineering team notices that each of the queries uses the same **SQL warehouse**, but the **SQL warehouse** is not used by any other user.

Which of the following approaches can the data engineering team use to improve the latency of the data analyst's queries?

- A. They can turn on the Serverless feature for the **SQL warehouse**.
- B. They can increase the maximum bound of the **SQL warehouse**'s scaling range.
- C. They can increase the cluster size of the **SQL warehouse**.
- D. They can turn on the Auto Stop feature for the **SQL warehouse**.
- E. They can turn on the Serverless feature for the **SQL warehouse** and change the Spot Instance Policy to "Reliability Optimized."

✓ সঠিক উত্তর: C

ব্যাখ্যা

প্রশ্নের দুটো গুরুত্বপূর্ণ তথ্য:

1. Query গুলো চলছে **একটার পর একটা (sequentially)**, একসাথে নয়
2. Endpoint **অন্য কেউ ব্যবহার করছে না**, অর্থাৎ ভিড় (concurrency) কোনো সমস্যা নয়

তাহলে সমস্যাটা হলো **প্রতিটা query নিজেই ধীরে চলছে**। একটা query দ্রুত চালাতে লাগে **বেশি শক্তিশালী compute**।

Cluster size (T-shirt size) বাড়ালে:

- প্রতিটা cluster-এ **বেশি worker / CPU / memory** থাকে
- প্রতিটা query আরও বেশি resource পায়
- ফলে **প্রতিটা query দ্রুত চলে** (latency কমে) ✓

Cluster Size: 2X-Small → X-Small → Small → Medium → Large → X-Large → ...
(কম শক্তি) (বেশি শক্তি)

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Serverless মূলত endpoint চালু হওয়ার সময় (startup time) কমায়, আর Databricks নিজে compute সামলায়। কিন্তু চালু হওয়ার পর একটা query-র গতি মূলত cluster size -এর উপর নির্ভর করে। প্রশ্নে startup নিয়ে কোনো কথা নেই, সমস্যা হলো query চলতে দেরি হওয়া।
B	Scaling range (min/max cluster) বাড়ালে একসাথে চলা অনেক query (concurrency) সামলানোর জন্য বাড়তি cluster যোগ হয়। কিন্তু এখানে query চলছে একটার পর একটা আর একজনই user, তাই একসাথে একটার বেশি cluster লাগেই না। বাড়তি cluster যোগ হলেও প্রতিটা query দ্রুত হবে না।
D	Auto Stop একটা নির্দিষ্ট সময় ব্যবহার না হলে endpoint বন্ধ করে দেয়, খরচ বাঁচানোর জন্য। এতে গতি বাড়ে না, বরং আবার চালু হতে সময় লাগায় latency বাড়তে পারে।
E	Serverless-এর কথা আগেই বলা হয়েছে। Spot Instance Policy ঠিক করে সস্তা spot instance ব্যবহার করা হবে কিনা। "Reliability Optimized" মানে বেশি on-demand instance, তাই instance হঠাৎ হারানোর ঝুঁকি কমে। কিন্তু এতে query-র গতি বাড়ে না ।

🔑 মনে রাখার উপায়

সমস্যা	সমাধান
প্রতিটা query ধীর (sequential, একজন user)	Cluster size বাড়ানো ✅ (Scale Up)
অনেক query একসাথে, query queue-তে আটকে থাকে (অনেক user)	Scaling range (max clusters) বাড়ানো (Scale Out)
Endpoint চালু হতে দেরি হয়	Serverless চালু করা
খরচ কমানো	Auto Stop চালু করা
Instance হঠাৎ হারানোর ঝুঁকি	Spot policy → Reliability Optimized

💡 **Cluster size = প্রতিটা query-র গতি** (vertical scaling) 💡 **Scaling range = একসাথে কয়টা query** (horizontal scaling)

সংক্ষেপে: একজন user-এর একটার পর একটা চলা query ধীর হলে **cluster size বাড়াতে** হবে, যাতে প্রতিটা query বেশি resource পায়। তাই উত্তর **C**।

Question 35

QUESTION

🚫 **OUT OF CURRENT SYLLABUS** – এই প্রশ্নটি বর্তমান syllabus (May 2026 exam guide)-এর বাইরে

কারণ: Databricks SQL query schedule / Alert / dashboard refresh – কোনো exam objective-এ নেই

An engineering manager uses a Databricks SQL query to monitor their team's progress on fixes related to customer-reported bugs. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.

Which of the following approaches can the manager use to ensure the results of the query are updated each day?

- A. They can schedule the query to run every 1 day from the **Lakeflow Jobs UI**.
- B. They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
- C. They can schedule the query to run every 12 hours from the **Lakeflow Jobs UI**.
- D. They can schedule the query to refresh every 1 day from the **SQL warehouse**'s page in Databricks SQL.
- E. They can schedule the query to refresh every 12 hours from the **SQL warehouse**'s page in Databricks SQL.

✅ **সঠিক উত্তর: B**

ব্যাখ্যা

Databricks SQL-এ প্রতিটা query-র নিজের page থেকেই সেটার refresh schedule সেট করা যায়। এজন্য আলাদা Job বানাতে হয় না।

কীভাবে সেট করতে হয়:

Databricks SQL → Queries → নির্দিষ্ট query খোলা → উপরে ডান দিকে "Schedule" →
Refresh every: 1 day → সময় ও timezone বেছে নেওয়া →
কোন SQL warehouse/endpoint-এ চলবে ঠিক করা → Save

এতে:

- Query প্রতিদিন নির্দিষ্ট সময়ে নিজে থেকে চলেবে
- Manager query খুললেই সর্বশেষ ফলাফল আগে থেকেই তৈরি পাবেন, অপেক্ষা করতে হবে না
- চাইলে ফলাফল email বা alert-এর মাধ্যমেও পাঠানো যায়

প্রশ্নে বলা আছে manager প্রতিদিন ফলাফল দেখেন, তাই ১ দিন পর পর refresh-ই যথেষ্ট।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Databricks SQL query-র refresh schedule করার সহজ ও স্বাভাবিক জায়গা হলো query-র নিজের page। Jobs UI মূলত notebook, DLT pipeline আর বড় workflow-এর জন্য। (নতুন version-এ Jobs-এ SQL task রাখা যায়, তবে পরীক্ষার প্রশ্নের প্রেক্ষিতে এটা প্রত্যাশিত উত্তর নয়।)
C	একই কারণে Jobs UI ঠিক জায়গা নয়। তাছাড়া ১২ ঘণ্টা পর পর অপ্রয়োজনীয়, কারণ manager দিনে একবার দেখেন। এতে বাড়তি compute খরচ হয়।
D	SQL endpoint (warehouse)-এর page-এ শুধু compute-এর setting (cluster size, scaling, auto stop ইত্যাদি) থাকে। এখান থেকে query schedule করা যায় না।
E	Endpoint-এর page থেকে schedule করা যায় না, আর ১২ ঘণ্টাও অপ্রয়োজনীয়। দুই দিক থেকেই ভুল।

🔑 মনে রাখার উপায়

কী schedule করতে চান	কোথা থেকে
Databricks SQL query	Query-র page → Schedule ✅
Dashboard	Dashboard-এর page → Schedule
Alert	Alert-এর page → refresh schedule
Notebook / DLT pipeline / multi-task workflow	Jobs (Workflows) UI

SQL Endpoint (Warehouse) page-এ কী থাকে
Cluster size, Scaling (min/max), Auto Stop, Serverless, Spot policy
❌ Query schedule নেই

💡 SQL query schedule = query-র নিজের page থেকে 💡 Endpoint = শুধু compute setting

সংক্ষেপে: Databricks SQL-এ query-র page থেকেই প্রতি ১ দিনে refresh schedule করা যায়। তাই উত্তর B।

Question 36

QUESTION

🚫 **OUT OF CURRENT SYLLABUS** — এই প্রশ্নটি বর্তমান syllabus (May 2026 exam guide)-এর বাইরে

কারণ: Databricks SQL query schedule / Alert / dashboard refresh — কোনো exam objective-এ নেই

A data engineering team has been using a Databricks SQL query to monitor the performance of an ELT job. The ELT job is triggered by a specific number of input records being ready to process. The Databricks SQL query returns the number of minutes since the job's most recent runtime.

Which of the following approaches can enable the data engineering team to be notified if the ELT job has not been run in an hour?

- A. They can set up an Alert for the accompanying dashboard to notify them if the returned value is greater than 60.
- B. They can set up an Alert for the query to notify when the ELT job fails.
- C. They can set up an Alert for the accompanying dashboard to notify when it has not refreshed in 60 minutes.
- D. They can set up an Alert for the query to notify them if the returned value is greater than 60.
- E. This type of alerting is not possible in Databricks.

✅ **সঠিক উত্তর: D**

ব্যাখ্যা

Databricks SQL Alert একটি **query**-র ফলাফলের উপর ভিত্তি করে কাজ করে। Query নির্দিষ্ট সময় পর পর চলে, আর ফলাফল একটি শর্ত পূরণ করলে notification পাঠায় (email, Slack, webhook ইত্যাদি)।

এখানে:

- Query ফেরত দেয় → **শেষ run-এর পর কত মিনিট পেরিয়েছে**
- এক ঘণ্টা = **60 মিনিট**
- তাই শর্ত হবে → **মান > 60** হলে alert

কীভাবে সেট করতে হয়:

Databricks SQL → Alerts → Create Alert →
Query: (ঐ monitoring query টা বেছে নেওয়া)
Trigger when: `minutes_since_last_run > 60`
Refresh: (যেমন প্রতি ১০ মিনিটে)
Notify: email / Slack / webhook

উদাহরণ query:

```
SELECT TIMESTAMPDIFF(MINUTE, MAX(run_time), CURRENT_TIMESTAMP()) AS minutes_since_last_run
FROM elt_job_runs;
```

Query-র ফলাফল	Alert-এর অবস্থা
25	✅ OK (৬০-এর কম)
58	✅ OK
75	🔔 TRIGGERED : notification যাবে

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Databricks SQL-এ Alert সেট করা হয় query-র উপর, dashboard-এর উপর নয় । শর্তটি (> 60) ঠিক, কিন্তু জায়গাটা ভুল।
B	প্রশ্নের সমস্যা হলো job এক ঘণ্টা চলেনি , job fail করেছে এমন নয়। Job না চললে fail-ও হয় না, তাই এই alert কখনো বাজবে না। তাছাড়া query শুধু "কত মিনিট" ফেরত দেয়, fail হয়েছে কিনা নয়।
C	Alert dashboard-এর উপর সেট করা যায় না । আর dashboard refresh হলো কিনা তার সাথে job চলল কিনা, তার কোনো সম্পর্ক নেই।
E	ভুল। Databricks SQL Alerts দিয়ে ঠিক এই ধরনের alert দেওয়া যায়।

🔑 মনে রাখার উপায়

Databricks SQL Alert-এর অংশ	এই প্রশ্নে
কিসের উপর	Query (dashboard নয়) ✅
কোন column	minutes since last run
শর্ত (operator)	>
Threshold মান	60
Notification	Email / Slack / Webhook / PagerDuty ইত্যাদি

💡 **Alert = Query + শর্ত + Notification** 💡 Alert সবসময় **query-র উপর** সেট হয়, **dashboard-এর উপর নয়**

সংক্ষেপে: **query-র উপর একটা Alert** সেট করে ফেরত আসা মান **60-এর বেশি** হলে notification পাঠানো যায়। তাই উত্তর **D**।

Question 37

QUESTION

🚫 **OUT OF CURRENT SYLLABUS** – এই প্রশ্নটি বর্তমান syllabus (May 2026 exam guide)-এর বাইরে

কারণ: Databricks SQL query schedule / Alert / dashboard refresh – কোনো exam objective-এ নেই

A data engineering manager has noticed that each of the queries in a Databricks SQL dashboard takes a few minutes to update when they manually click the "Refresh" button. They are curious why this might be occurring, so a team member provides a variety of reasons on why the delay might be occurring.

Which of the following reasons fails to explain why the dashboard might be taking a few minutes to update?

- A. The **SQL warehouse** being used by each of the queries might need a few minutes to start up.
- B. The queries attached to the dashboard might take a few minutes to run under normal circumstances.
- C. The queries attached to the dashboard might first be checking to determine if new data is available.
- D. The Job associated with updating the dashboard might be using a non-pooled endpoint.
- E. The queries attached to the dashboard might all be connected to their own, unstarted Databricks clusters.

✅ **সঠিক উত্তর (যে কারণ ব্যাখ্যা দিতে পারে না): D**

⚠️ খেয়াল করুন: প্রশ্নে এমন কারণ খুঁজতে বলা হয়েছে যেটা দেবির **ব্যাখ্যা দেয় না**।

ব্যাখ্যা

D কেন দেবির ব্যাখ্যা দিতে পারে না? এখানে দুটো সমস্যা আছে:

1. **কোনো Job জড়িত নেই।** Manager নিজে হাতে **"Refresh" button চাপছেন।** এতে dashboard-এর query গুলো সরাসরি SQL endpoint-এ চলে, কোনো **Job চালু হয় না।** তাই "Job কোন endpoint ব্যবহার করছে" সেটা এখানে প্রাসঙ্গিকই নয়।
2. **"Non-pooled endpoint" বলে কিছু নেই।** **Pool** (instance pool) হলো সাধারণ Databricks **cluster**-এর একটা সুবিধা। Databricks SQL endpoint (warehouse)-এ "pooled" বা "non-pooled" বলে কোনো ধারণা নেই।

তাই এই কারণ দিয়ে দেবি ব্যাখ্যা করা যায় না।

✓ বাকি কারণগুলো কেন দেবির ব্যাখ্যা দিতে পারে

Option	কেন সম্ভাব্য কারণ
A	SQL endpoint বন্ধ (stopped) থাকলে, যেমন Auto Stop-এর পরে, query চালানোর আগে endpoint চালু হতে কয়েক মিনিট লাগে (Serverless না হলে)। এটা দেবির খুব সাধারণ কারণ।
B	Query গুলো যদি বড় data নিয়ে বা জটিল হিসাব (join, aggregation) করে, তাহলে স্বাভাবিকভাবেই চলতে কয়েক মিনিট লাগতে পারে।
C	Query চালানোর আগে নতুন data আছে কিনা দেখতে (যেমন metadata বা file listing পড়তে) বাড়তি সময় লাগতে পারে।
E	Query যদি বন্ধ থাকা cluster -এর সাথে যুক্ত থাকে, তাহলে আগে cluster চালু হতে হবে , যাতে কয়েক মিনিট লাগে। (Databricks SQL সাধারণত SQL endpoint-এ চলে, কিন্তু যুক্তি হিসেবে "বন্ধ compute চালু হতে সময় নেয়" কথাটা দেবি ব্যাখ্যা করে।)

🔑 মনে রাখার উপায়

Dashboard দেবিতে update হওয়ার আসল কারণগুলো:

কারণ	সমাধান
Endpoint বন্ধ ছিল, চালু হতে সময় লাগে	Serverless চালু করা, বা Auto Stop-এর সময় বাড়ানো
Query নিজেই ভারী	Query optimize করা, cluster size বাড়ানো
অনেক query একসাথে চলে, queue-তে আটকে থাকে	Scaling range (max clusters) বাড়ানো

যেটা ভুল ধারণা:

ধারণা	কেন ভুল
Manual refresh-এ Job চলে	✗ Manual refresh-এ সরাসরি query চলে, Job নয়
SQL endpoint pooled/non-pooled	✗ Pool হলো cluster-এর সুবিধা, SQL endpoint-এর নয়

সংক্ষেপে: নিজে হাতে refresh করলে কোনো **Job** চলে না, আর SQL endpoint-এ "**non-pooled**" বলে কিছু নেই। তাই দেবির ব্যাখ্যা দিতে পারে না **D**।

Question 38

QUESTION

A new data engineer has started at a company. The data engineer has recently been added to the company's Databricks workspace as new.engineer@company.com. The data engineer needs to be able to query the table sales in the database retail. The new data engineer already has been granted **USE SCHEMA** on the database retail.

Which of the following commands can be used to grant the appropriate permissions to the new data engineer?

- A. GRANT USAGE ON TABLE sales TO new.engineer@company.com;
- B. GRANT CREATE ON TABLE sales TO new.engineer@company.com;
- C. GRANT SELECT ON TABLE sales TO new.engineer@company.com;
- D. GRANT USAGE ON TABLE new.engineer@company.com TO sales;
- E. GRANT SELECT ON TABLE new.engineer@company.com TO sales;

✓ সঠিক উত্তর: C

```
GRANT SELECT ON TABLE sales TO new.engineer@company.com;
```

ব্যাখ্যা

একটা table-এ query চালাতে (data পড়তে) দুটো permission লাগে:

ধাপ	Permission	কিসের উপর	অবস্থা
১	USAGE	Database (retail)	✓ আগেই দেওয়া আছে
২	SELECT	Table (sales)	✗ এখনো বাকি → এটাই দিতে হবে

- **USAGE** → database-এর ভেতরে ঢোকার অনুমতি (দরজা খোলা)
- **SELECT** → table-এর data পড়ার অনুমতি (query চালানো)

USAGE আগেই দেওয়া আছে, তাই এখন শুধু table-এ **SELECT** দিলেই data engineer query চালাতে পারবেন।

GRANT command-এর গঠন:

```
GRANT <privilege> ON <object_type> <object_name> TO <user/group>;  
SELECT      ON TABLE      sales              TO new.engineer@company.com;
```

💡 বাস্তবে email-এ . আর @ থাকায় user-এর নাম **backtick** দিয়ে লিখতে হয়:

```
GRANT SELECT ON TABLE retail.sales TO `new.engineer@company.com`;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	USAGE দেওয়া হয় database/schema/catalog -এর উপর, ঢাকার অনুমতি হিসেবে। Table-এ USAGE দিলে data পড়ার অনুমতি পাওয়া যায় না। Data পড়তে লাগে SELECT ।
B	CREATE দেওয়া হয় নতুন object (table/view) বানানোর জন্য, data পড়ার জন্য নয়। আর table-এর উপর CREATE অর্থহীন।
D	ক্রম উল্টো : ON -এর পরে বসে object (table), আর TO -এর পরে বসে user । এখানে user আর table জায়গা বদল করেছে। তার উপর USAGE ভুল privilege।
E	SELECT ঠিক, কিন্তু ক্রম উল্টো (user আর table জায়গা বদল করেছে)। Syntax ভুল।

🔑 মনে রাখার উপায়

গুরুত্বপূর্ণ privilege গুলো:

Privilege	কাজ	সাধারণত কিসের উপর
USAGE	Database/schema-তে ঢাকার অনুমতি	Database / Schema / Catalog
SELECT	Data পড়া (query) ✅	Table / View
MODIFY	Data যোগ / বদল / মোছা (INSERT, UPDATE, DELETE)	Table
CREATE	নতুন object বানানো	Database / Schema
READ_METADATA	Table-এর metadata (schema) দেখা	Table
ALL PRIVILEGES	সব অনুমতি	যেকোনো

Table-এ query চালানোর জন্য:

USAGE (database-এ) + SELECT (table-এ) = ✅ Query চালানো যায়

Syntax মনে রাখার উপায়:

GRANT [কী অনুমতি] ON [কিসের উপর] TO [কাকে]

💡 Unity Catalog-এ **USAGE** -এর বদলে **USE CATALOG** আর **USE SCHEMA** ব্যবহার হয়। পরীক্ষার প্রশ্নে পুরনো Hive metastore-এর **USAGE** দেখা যায়।

সংক্ষেপে: database-এ USAGE আগেই আছে, এখন table-এ data পড়তে **GRANT SELECT ON TABLE sales TO ...** লাগবে। তাই উত্তর **C**।

Question 39

QUESTION

A new data engineer new.engineer@company.com has been assigned to an ELT project. The new data engineer will need full privileges on the table sales to fully manage the project.

Which of the following commands can be used to grant full permissions on the table to the new data engineer?

- A. GRANT ALL PRIVILEGES ON TABLE sales TO new.engineer@company.com;
- B. GRANT USAGE ON TABLE sales TO new.engineer@company.com;
- C. GRANT ALL PRIVILEGES ON TABLE new.engineer@company.com TO sales;
- D. GRANT SELECT ON TABLE sales TO new.engineer@company.com;
- E. GRANT SELECT CREATE MODIFY ON TABLE sales TO new.engineer@company.com;

✓ সঠিক উত্তর: A

```
GRANT ALL PRIVILEGES ON TABLE sales TO new.engineer@company.com;
```

ব্যাখ্যা

ALL PRIVILEGES একটি object-এর উপর **প্রয়োজ্য সব privilege একসাথে** দিয়ে দেয়। একটি একটি করে আলাদা privilege লিখতে হয় না।

Table-এর ক্ষেত্রে **ALL PRIVILEGES** -এর মধ্যে থাকে:

Privilege	কাজ
SELECT	Data পড়া
MODIFY	Data যোগ / বদল / মোছা (INSERT, UPDATE, DELETE, MERGE)
READ_METADATA	Table-এর schema/metadata দেখা
(ইত্যাদি)	Table-এর উপর প্রয়োজ্য বাকি সব privilege

Command-এর গঠনও ঠিক আছে:

```
GRANT  ALL PRIVILEGES  ON  TABLE sales  TO  new.engineer@company.com;  
[কী অনুমতি]           [কিসের উপর]           [কাকে]
```

💡 বাস্তবে email-এ `.` আর `@` থাকায় নামটা **backtick** দিয়ে লিখতে হয়।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B	USAGE শুধু database/schema-তে টোকার অনুমতি। Table-এর উপর সব privilege দেয় না।
C	ALL PRIVILEGES ঠিক, কিন্তু ক্রম উল্টো : ON -এর পরে বসবে table, আর TO -এর পরে user। এখানে দুটো জায়গা বদল করেছে।
D	SELECT শুধু data পড়ার অনুমতি দেয়। Data বদলানো বা project পুরোপুরি সামলানোর জন্য এটা যথেষ্ট নয়।
E	একাধিক privilege লিখলে কমা (,) দিয়ে আলাদা করতে হয় (SELECT, MODIFY), তাই এই syntax ভুল। তাছাড়া table-এর উপর CREATE প্রযোজ্য নয় (CREATE দেওয়া হয় database/schema-তে), আর এতে সব privilege-ও আসে না।

🔑 মনে রাখার উপায়

প্রয়োজন	Command
Table-এ সব অনুমতি	GRANT ALL PRIVILEGES ON TABLE t TO user ✅
Table-এ শুধু পড়া	GRANT SELECT ON TABLE t TO user
Table-এ পড়া + লেখা	GRANT SELECT, MODIFY ON TABLE t TO user (কমা দিয়ে)
Database-এ টোকা	GRANT USAGE ON DATABASE db TO user
Database-এ নতুন table বানানো	GRANT CREATE ON DATABASE db TO user
অনুমতি ফেরত নেওয়া	REVOKE SELECT ON TABLE t FROM user
কার কী অনুমতি আছে দেখা	SHOW GRANTS ON TABLE t

💡 "Full privileges" = **ALL PRIVILEGES** 💡 একাধিক privilege = কমা দিয়ে আলাদা: **SELECT, MODIFY** 💡
ক্রম: **GRANT ... ON [object] TO [user]**

সংক্ষেপে: table-এর উপর সব permission একসাথে দিতে **GRANT ALL PRIVILEGES ON TABLE sales TO ...** লিখতে হয়।
তাই উত্তর **A**।

Question 40

QUESTION

Which of the following describes a benefit of a data lakehouse that is unavailable in a traditional data warehouse?

- A. A data lakehouse provides a relational system of data management.
- B. A data lakehouse captures snapshots of data for version control purposes.
- C. A data lakehouse couples storage and compute for complete control.
- D. A data lakehouse utilizes proprietary storage formats for data.
- E. A data lakehouse enables both batch and streaming analytics.

✓ সঠিক উত্তর: E

ব্যাখ্যা

Data Lakehouse হলো **Data Lake**-এর নমনীয়তা আর কম খরচ, এবং **Data Warehouse**-এর নির্ভরযোগ্যতা আর performance একসাথে নিয়ে আসা একটা architecture।

Batch + Streaming কেন lakehouse-এর বিশেষ সুবিধা?

- **Traditional data warehouse** মূলত **batch processing**-এর জন্য বানানো। Data সাধারণত নির্দিষ্ট সময় পর পর (যেমন রাতে একবার) ETL করে load হয়। Real-time streaming data সরাসরি সামলানোর ক্ষমতা সাধারণত থাকে না।
- **Data lakehouse**-এ (যেমন Databricks + Delta Lake) **একই table-এ batch আর streaming দুটোই** চালানো যায়:
 - Spark Structured Streaming দিয়ে real-time data ingest
 - একই Delta table-এ batch query
 - আলাদা system লাগে না

```
# Streaming write
(spark.readStream.format("cloudFiles")...
  .writeStream.table("sales"))

# একই table-এ batch read
spark.table("sales").groupBy("region").sum("amount")
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Relational data management (table, schema, SQL) traditional data warehouse-এর মূল বৈশিষ্ট্য। তাই এটা lakehouse-এর বাড়তি সুবিধা নয়।
B	Snapshot / version রাখা (Delta Lake Time Travel) lakehouse-এ আছে, কিন্তু অনেক আধুনিক data warehouse-এও এমন সুবিধা আছে (যেমন snapshot, time travel)। তাই এটাকে "warehouse-এ পাওয়া যায় না" বলা যায় না।
C	উল্টো। Lakehouse storage আর compute আলাদা (decoupled) রাখে। Storage থাকে cloud object storage-এ (S3, ADLS, GCS), আর compute দরকার মতো বাড়ানো-কমানো যায়। Storage আর compute একসাথে জুড়ে রাখা traditional warehouse-এর বৈশিষ্ট্য।
D	উল্টো। Lakehouse open format (Delta Lake, Parquet) ব্যবহার করে, যাতে কোনো নির্দিষ্ট vendor-এর উপর আটকে থাকতে না হয়। Proprietary format traditional warehouse-এর বৈশিষ্ট্য।

🔑 মনে রাখার উপায়

বৈশিষ্ট্য	Traditional Data Warehouse	Data Lakehouse
Data format	Proprietary (বন্ধ)	Open (Delta, Parquet)
Storage + Compute	একসাথে জোড়া (coupled)	আলাদা (decoupled)
Data-র ধরন	মূলত structured	Structured + semi-structured + unstructured
Processing	মূলত batch	Batch + Streaming ✅
Workload	মূলত BI / SQL	BI + SQL + ML / Data Science
খরচ	বেশি	কম (cloud object storage)
ACID transaction	✅	✅ (Delta Lake দিয়ে)

💡 **Lakehouse = Data Lake (open, সস্তা, সব ধরনের data) + Data Warehouse (ACID, performance, governance)**

সংক্ষেপে: একই platform-এ **batch আর streaming দুই ধরনের analytics** চালাতে পারা data lakehouse-এর সুবিধা, যা traditional data warehouse-এ সাধারণত পাওয়া যায় না। তাই উত্তর **E**।

Question 41

QUESTION

Which of the following locations hosts the driver and worker nodes of a Databricks-managed cluster?

- A. Data plane
- B. Control plane
- C. Databricks Filesystem
- D. JDBC data source
- E. Databricks web application

✓ সঠিক উত্তর: A. Data plane

ব্যাখ্যা

Databricks-এর architecture দুটো প্রধান অংশে ভাগ করা:

CONTROL PLANE (Databricks-এর নিজের cloud account-এ)

- Web application (UI)
- Notebook & command
- Job scheduler
- Cluster manager
- Workspace configuration

▼ নির্দেশ পাঠায়

DATA PLANE (Compute Plane) (Customer-এর নিজের cloud account-এ)

- Cluster-এর **Driver node** ✓
- Cluster-এর **Worker node** ✓
- Data processing
- DBFS root storage / customer data

Data plane (নতুন নাম **Compute plane**) হলো সেই জায়গা, যেখানে **আসল data process হয়**। Cluster-এর **driver আর worker node** গুলো (অর্থাৎ VM/virtual machine) এখানেই চলে। Classic deployment-এ এগুলো **customer-এর নিজের cloud account-এ** (AWS, Azure, GCP) থাকে।

এর সুবিধা হলো, customer-এর **data তাঁর নিজের cloud account-এর বাইরে যায় না**, তাই নিরাপত্তা বেশি।

💡 **Serverless compute**-এর ক্ষেত্রে compute চলে Databricks-এর account-এর ভেতরে একটা serverless compute plane-এ। কিন্তু পরীক্ষার প্রশ্নের মূল কথা হলো: **cluster node = data plane**।

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
B. Control plane	Control plane Databricks-এর নিজের cloud account-এ থাকে। এখানে web UI, notebook, job scheduler আর cluster manager থাকে। Control plane cluster চালু/বন্ধ করার নির্দেশ দেয় , কিন্তু cluster-এর node গুলো এখানে চলে না ।
C. Databricks Filesystem (DBFS)	DBFS হলো একটা storage layer (file system), যা cloud object storage-এর উপর বসানো। এখানে data/file রাখা হয় , কিন্তু compute node চলে না।
D. JDBC data source	JDBC হলো বাইরের database-এর সাথে সংযোগ করার উপায় (যেমন MySQL, PostgreSQL)। এটা data-র উৎস, cluster চালানোর জায়গা নয়।
E. Databricks web application	Web application হলো user interface , যেখানে notebook লেখা, job বানানো ইত্যাদি হয়। এটা control plane-এর অংশ , compute node এখানে চলে না।

🔑 মনে রাখার উপায়

Control Plane	Data Plane (Compute Plane)
Databricks-এর cloud account-এ	Customer-এর cloud account-এ
Web UI / Web application	Driver node ✅
Notebook (code)	Worker node ✅
Job scheduler	Data processing
Cluster manager	Customer data / storage
"মস্তিষ্ক": কী করতে হবে ঠিক করে	"হাত-পা": আসল কাজ করে

💡 **Control plane = নিয়ন্ত্রণ (Databricks-এর)** 💡 **Data plane = compute + data (customer-এর)**

সংক্ষেপে: cluster-এর **driver আর worker node** গুলো চলে **Data plane-এ**। তাই উত্তর **A**।

Question 42

QUESTION

A data architect is designing a data model that works for both video-based machine learning workloads and highly audited batch ETL/ELT workloads.

Which of the following describes how using a data lakehouse can help the data architect meet the needs of both workloads?

- A. A data lakehouse requires very little data modeling.
- B. A data lakehouse combines compute and storage for simple governance.
- C. A data lakehouse provides autoscaling for compute clusters.
- D. A data lakehouse stores unstructured data and is ACID-compliant.
- E. A data lakehouse fully exists in the cloud.

✓ সঠিক উত্তর: D

ব্যাখ্যা

এখানে দুটো আলাদা workload, আর প্রতিটির আলাদা চাহিদা আছে:

Workload	কী দরকার	Lakehouse কীভাবে মেটায়
 Video-ভিত্তিক ML	Unstructured data (video, image, audio) রাখতে পারা	Lakehouse cloud object storage-এর উপর বানানো, তাই যেকোনো ধরনের data রাখা যায় (data lake-এর সুবিধা)
 কঠোরভাবে audit করা batch ETL/ELT	নির্ভরযোগ্যতা, data-র সঠিকতা, পরিবর্তনের হিসাব	Delta Lake দিয়ে ACID transaction , transaction log, DESCRIBE HISTORY , time travel (data warehouse-এর সুবিধা)

Option D দুটো চাহিদাই একসাথে মেটায়:

- "Stores unstructured data" → video ML workload-এর জন্য ✓
- "ACID-compliant" → audit করা ETL workload-এর জন্য ✓

কেন অন্য architecture এটা পারে না?

Architecture	Unstructured data	ACID / Audit
Data Lake	✓	✗ (ACID নেই, data নষ্ট হতে পারে)
Data Warehouse	✗ (মূলত structured)	✓
Data Lakehouse	✓	✓

Audit-এর জন্য Delta Lake-এর সুবিধা:

```
-- প্রতিটা পরিবর্তনের ইতিহাস (কে, কখন, কী করেছে)
DESCRIBE HISTORY etl_table;

-- আগের অবস্থার data দেখা
SELECT * FROM etl_table VERSION AS OF 5;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Lakehouse-এ data modeling কম লাগে, এমন নয় । Medallion architecture (Bronze/Silver/Gold), schema design ইত্যাদি এখনো দরকার। আর এটা দুই workload-এর চাহিদার সাথে সম্পর্কিতও নয়।
B	উল্টো। Lakehouse storage আর compute আলাদা (decoupled) রাখে। একসাথে জুড়ে রাখা traditional data warehouse-এর বৈশিষ্ট্য।
C	Autoscaling একটা ভালো সুবিধা, কিন্তু এটা cluster/compute-এর বৈশিষ্ট্য, data model-এর নয়। Unstructured data রাখা বা audit করার চাহিদা এতে মেটে না।
E	Cloud-এ থাকা lakehouse-এর একার সুবিধা নয় । অনেক data warehouse-ও cloud-এ চলে। আর এতে দুই workload-এর নির্দিষ্ট চাহিদাও মেটে না।

🔑 মনে রাখার উপায়

চাহিদা	Lakehouse-এর কোন বৈশিষ্ট্য
Video, image, audio (ML)	Unstructured data support (data lake থেকে)
Audit, নির্ভরযোগ্যতা, data-র সঠিকতা	ACID transaction + transaction log (Delta Lake)
পরিবর্তনের ইতিহাস	<code>DESCRIBE HISTORY</code> / Time Travel
BI ও SQL	Data warehouse-এর মতো performance

💡 Lakehouse = সব ধরনের data (Lake) + ACID ও নির্ভরযোগ্যতা (Warehouse) 💡 ML + Audit একসাথে = **Unstructured + ACID**

সংক্ষেপে: lakehouse **unstructured data** (video ML-এর জন্য) রাখতে পারে, আবার **ACID-compliant**-ও (audit করা ETL-এর জন্য)। তাই উত্তর **D**।

Question 43

QUESTION

Which of the following describes a scenario in which a data engineer will want to use a Job cluster instead of an all-purpose cluster?

- A. An ad-hoc analytics report needs to be developed while minimizing compute costs.
- B. A data team needs to collaborate on the development of a machine learning model.
- C. An automated workflow needs to be run every 30 minutes.
- D. A Databricks SQL query needs to be scheduled for upward reporting.
- E. A data engineer needs to manually investigate a production error

✓ সঠিক উত্তর: C

ব্যাখ্যা

Databricks-এ দুই ধরনের cluster আছে:

বৈশিষ্ট্য	All-purpose cluster	Job cluster
কীভাবে বানানো হয়	UI / API দিয়ে নিজে হাতে	Job চালু হলে নিজে থেকেই তৈরি হয়
কতক্ষণ চলে	নিজে বন্ধ না করা পর্যন্ত (বা auto-terminate)	Job শেষ হলেই বন্ধ
কে ব্যবহার করে	একাধিক user একসাথে (shared)	শুধু সেই নির্দিষ্ট Job
কিসের জন্য	Interactive কাজ: notebook-এ code লেখা, explore, debug	Automated কাজ: schedule করা job, production pipeline
খরচ	বেশি (DBU rate বেশি)	কম (DBU rate কম)

Option C কেন Job cluster-এর জন্য উপযুক্ত?

- **Automated workflow** → মানুষের হাত লাগে না, Job নিজে চলে
- **প্রতি ৩০ মিনিটে** → schedule করা কাজ
- প্রতিবার Job চালু হলে cluster তৈরি হয়, কাজ শেষ হলে **নিজে থেকেই বন্ধ** হয়ে যায়
- Job cluster-এর DBU rate কম, তাই **খরচও কম**
- প্রতিটা run একটা পরিষ্কার, আলাদা cluster-এ চলে (**isolation**), আগের কোনো অবস্থা থেকে যায় না

প্রতি ৩০ মিনিটে:

Job trigger → Job cluster তৈরি → কাজ চলে → Job শেষ → Cluster বন্ধ 💰

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A	Ad-hoc analytics মানে notebook-এ বসে interactive ভাবে explore করা আর report বানানো। এটা interactive কাজ, তাই all-purpose cluster লাগে। খরচের কথা বলা থাকলেও কাজের ধরন interactive।
B	একাধিক মানুষ একসাথে মিলে model বানানো মানে shared, interactive কাজ। এর জন্য all-purpose cluster উপযুক্ত, কারণ অনেক user একসাথে এটায় attach করতে পারে।
D	Databricks SQL query চলে SQL endpoint / SQL warehouse -এ, Job cluster-এ নয়। Schedule-ও করা হয় query-র page থেকে।
E	নিজে হাতে error খতিয়ে দেখা (debug) একটা interactive কাজ। Notebook-এ বসে code চালিয়ে দেখতে হয়, তাই all-purpose cluster লাগে।

🔑 মনে রাখার উপায়

কাজের ধরন	কোন compute
Automated / scheduled job , production pipeline	Job cluster ✅
Interactive notebook, explore, develop, debug	All-purpose cluster
একাধিক মানুষ একসাথে কাজ (collaboration)	All-purpose cluster
Databricks SQL query / dashboard	SQL warehouse (endpoint)

💡 **Automated = Job cluster** (সস্তা, কাজ শেষে নিজে বন্ধ) 💡 **Interactive = All-purpose cluster** (shared, নিজে হাতে চালানো) 💡 **SQL = SQL warehouse**

সংক্ষেপে: প্রতি ৩০ মিনিটে চলা **automated workflow**-এর জন্য **Job cluster** সবচেয়ে উপযুক্ত, কারণ এটা কাজ শেষে নিজে বন্ধ হয় আর খরচও কম। তাই উত্তর **C**।

Question 44

QUESTION

A data engineer has created a Delta table as part of a data pipeline. Downstream data analysts now need SELECT permission on the Delta table.

Assuming the data engineer is the Delta table owner, which part of the **Databricks Data Intelligence Platform** can the data engineer use to grant the data analysts the appropriate access?

- A. **Git Folders**
- B. Jobs
- C. **Catalog Explorer**
- D. Databricks Filesystem
- E. Dashboards

✅ সঠিক উত্তর: C. Data Explorer

ব্যাখ্যা

Data Explorer (নতুন নাম **Catalog Explorer**) হলো Databricks-এর একটা UI, যেখান থেকে database, table, view আর তাদের **permission** দেখা ও সামলানো যায়।

Data Explorer দিয়ে যা করা যায়:

- Database / schema / table / view খুঁজে দেখা
- Table-এর **schema, sample data, history, details** দেখা
- Table-এর **owner** দেখা ও বদলানো
- **Permission দেওয়া (GRANT) ও ফেরত নেওয়া (REVOKE)** ✅

কীভাবে SELECT permission দিতে হয়:

```
Sidebar → Data (Catalog) → database বেছে নেওয়া → table বেছে নেওয়া →  
Permissions tab → Grant →  
  User/Group: data_analysts  
  Privilege: ☒ SELECT  
→ Grant
```

Owner হিসেবে data engineer-এর অন্যদের permission দেওয়ার অধিকার আছে।

SQL দিয়ে একই কাজ:

```
GRANT SELECT ON TABLE my_delta_table TO `data_analysts`;
```

❌ বাকি option গুলো কেন ভুল

Option	কেন ভুল
A. Repos	Git-এর সাথে যুক্ত হয়ে code (notebook, file) version control করার জন্য। Table-এর permission এখান থেকে সামলানো যায় না।
B. Jobs	Notebook, pipeline বা workflow schedule ও automate করার জন্য। Job-এর নিজের permission (কে job চালাতে/দেখতে পারবে) সেট করা যায়, কিন্তু table-এর permission নয় ।
D. Databricks Filesystem (DBFS)	File রাখার storage layer । এখানে file থাকে, কিন্তু table-এ SQL permission (SELECT ইত্যাদি) দেওয়ার কোনো ব্যবস্থা নেই।
E. Dashboards	Query-র ফলাফল visualization আকারে দেখানোর জন্য। Dashboard share করা যায়, কিন্তু সেটা table-এর SELECT permission দেওয়া নয় ।

🔑 মনে রাখার উপায়

Databricks-এর অংশ	মূল কাজ
Data Explorer	Database/table দেখা + permission সামলানো ✅
Repos	Code-এর Git version control
Jobs / Workflows	কাজ schedule ও automate করা
DBFS	File storage
Dashboards	Visualization ও report
Compute	Cluster ও SQL warehouse

Table-এ permission দেওয়ার দুই উপায়:

উপায়	কীভাবে
UI	Data Explorer → table → Permissions → Grant ✅
SQL	<code>GRANT SELECT ON TABLE t TO user_or_group</code>

💡 **Table-এর permission = Data Explorer (UI দিয়ে) অথবা GRANT (SQL দিয়ে)**

সংক্ষেপে: table-এর permission দেখা ও দেওয়ার UI হলো **Data Explorer**। তাই উত্তর **C**।

Question 45

QUESTION

🚫 **OUT OF CURRENT SYLLABUS** — এই প্রশ্নটি বর্তমান syllabus (May 2026 exam guide)-এর বাইরে
কারণ: Notebook features (co-authoring, magic command, cell language) — কোনো exam objective-এ নেই

Two junior data engineers are authoring separate parts of a single data pipeline notebook. They are working on separate Git branches so they can pair program on the same notebook simultaneously. A senior data engineer experienced in Databricks suggests there is a better alternative for this type of collaboration.

Which of the following supports the senior data engineer's claim?

- A. Databricks Notebooks support automatic change-tracking and versioning
- B. Databricks Notebooks support real-time coauthoring on a single notebook
- C. Databricks Notebooks support commenting and notification comments
- D. Databricks Notebooks support the use of multiple languages in the same notebook
- E. Databricks Notebooks support the creation of interactive data visualizations

✅ সঠিক উত্তর: B

ব্যাখ্যা

সমস্যাটা কী? দুজন একই notebook-এ একসাথে কাজ করতে চান, কিন্তু আলাদা Git branch ব্যবহার করছেন। এতে:

- একজন আরেকজনের পরিবর্তন **সাথে সাথে দেখতে পান না**
- পরে branch **merge** করতে হয়
- Merge-এর সময় **conflict** হতে পারে, বিশেষ করে একই notebook-এ
- Pair programming-এর মূল উদ্দেশ্য (একসাথে বসে কাজ করা) পূরণ হয় না

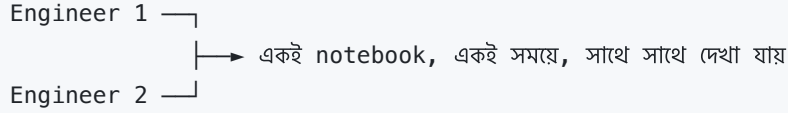
Real-time coauthoring কীভাবে সমাধান করে? Databricks Notebook-এ **Google Docs-এর মতো** একাধিক user **একই সময়ে একই notebook** খুলে কাজ করতে পারেন:

- প্রত্যেকের **cursor ও পরিবর্তন সাথে সাথে** অন্যরা দেখতে পান
- কে কোন cell-এ কাজ করছেন, সেটা দেখা যায়
- আলাদা branch, merge বা conflict-এর ঝামেলা নেই
- সত্যিকারের **pair programming** সম্ভব হয়

❌ আগে (আলাদা Git branch):



✅ পরে (Real-time coauthoring):



❌ বাকি option গুলো কেন ভুল

এগুলো সবই Databricks Notebook-এর **সত্যিকারের সুবিধা**, কিন্তু **একসাথে real-time-এ কাজ করার** সমস্যার সমাধান নয়:

Option	কেন এই পরিস্থিতিতে উত্তর নয়
A	Notebook-এ revision history নিজে থেকে থাকে, তাই আগের version দেখা বা সেখানে ফেরা যায়। কিন্তু এটা দুজনের একসাথে কাজ করার সমস্যা মেটায় না।
C	Comment দিয়ে মতামত দেওয়া ও আলোচনা করা যায়। এটা collaboration-এ সাহায্য করে, কিন্তু একই সময়ে একসাথে code লেখা (pair programming) এর মাধ্যমে হয় না।
D	<code>%python</code> , <code>%sql</code> , <code>%scala</code> , <code>%r</code> দিয়ে একই notebook-এ একাধিক ভাষা ব্যবহার করা যায়। কিন্তু এটা collaboration-এর সাথে সম্পর্কিত নয়।
E	Notebook-এ chart/graph বানানো যায়। এটাও collaboration-এর সাথে সম্পর্কিত নয়।

🔑 মনে রাখার উপায়

প্রয়োজন	Databricks-এর সুবিধা
একই notebook-এ একসাথে, একই সময়ে কাজ	Real-time coauthoring ✅
আগের version দেখা / ফিরে যাওয়া	Revision history (auto versioning)
Code-এর Git version control, CI/CD	Repos
মতামত দেওয়া / আলোচনা	Comments
একাধিক ভাষা	Magic commands (<code>%sql</code> , <code>%python</code>)

💡 "Simultaneously" / "pair program" / "same notebook" → Real-time coauthoring 💡 Git branch বেশি উপযুক্ত **আলাদা আলাদা feature-এ, আলাদা সময়ে** কাজ করার জন্য

সংক্ষেপে: একই notebook-এ একসাথে কাজ করতে আলাদা Git branch লাগে না। Databricks Notebook-এ **real-time coauthoring** আছে। তাই উত্তর B।

উত্তর সংক্ষেপ (Answer Key)

#	বিষয়	উত্তর
1	Databricks Repos ও CI/CD	E
2	Delta Lake কী	B
3	CREATE OR REPLACE TABLE (খালি Delta table)	B
4	INSERT INTO দিয়ে append	C
5	Z-Ordering	B
6	CREATE DATABASE IF NOT EXISTS ... LOCATION	C
7	Managed table	E
8	View (data কপি ছাড়া)	A
9	Parquet cache সমস্যা → Delta	A
10	CTAS-এ schema	A
11	Overwrite বনাম Delete + Recreate	B
12	SELECT DISTINCT	C
13	INNER JOIN	A
14	explode()	D
15	Struct field: dot notation	B
16	PySpark দিয়ে dynamic table নাম	A
17	createOrReplaceTempView()	A
18	Python f-string	D
19	readStream (Auto Loader)	A
20	trigger(once=True)	A
21	Auto Loader: শুধু নতুন file	E
22	format("cloudFiles")	C
23	Bronze table source	E
24	Silver table source	D
25	Bronze → Silver hop	C
26	DLT: table dependency	A
27	DLT pipeline: Jobs page	B
28	CREATE LIVE TABLE	B
29	DLT Expectations (warn)	A

30	DLT: Triggered + Development mode	D
31	Task-level retry policy	D
32	Task dependency (linear)	A
33	Job config: JSON	C
34	SQL endpoint: cluster size	C
35	SQL query schedule	B
36	Databricks SQL Alert	D
37	Dashboard refresh দেরি	D
38	GRANT SELECT	C
39	GRANT ALL PRIVILEGES	A
40	Lakehouse বনাম Data Warehouse	E
41	Control plane বনাম Data plane	A
42	Unstructured data + ACID	D
43	Job cluster বনাম All-purpose cluster	C
44	Data Explorer দিয়ে permission	C
45	Real-time coauthoring	B